Here's a procedure for building this agentic DWSIM flowsheet generator:

Phase 1 — Establish the DWSIM scripting foundation

1. Pick the DWSIM interface. DWSIM offers two automation paths:
   - DWSIM Automation (external, via `pythonnet` against the .NET libs) — best for an agent-driven workflow since you control the Python process.
   - Built-in Script Manager (IronPython inside DWSIM) — simpler but harder to drive programmatically.

   Recommend Automation.
2. Manually build 5–10 reference flowsheets in Python (flash drum → distillation → reactor → recycle loop → full ammonia loop). These become your gold-standard examples and test set. You can't design the agent until you know exactly what "correct" code looks like.
3. Document the API surface you'll actually use: `Simulation` object, `AddObject` for unit ops, `ConnectObjects` for streams, property package setup, compound selection, solver calls. Most of DWSIM's API is irrelevant for flowsheet generation — narrow it.

Phase 2 — Build the knowledge layer

4. Curated API reference (markdown/JSON): every unit op class, required parameters, valid connection topology. This is what the LLM consults — not the full DWSIM docs.
5. Process recipe library: canonical block diagrams for common processes (ammonia, methanol, ethylene, distillation trains). The LLM uses these as templates rather than inventing topology from scratch.
6. Component/property-package mapping: which thermodynamic package fits which chemistry (e.g., Peng-Robinson for hydrocarbons, NRTL for polar mixtures).

Phase 3 — Agent architecture

7. Decompose into tools, not one big prompt. Give the LLM function-calling tools like `list_unit_ops`, `get_unit_op_schema`, `add_unit`, `connect_streams`, `set_property_package`, `run_simulation`, `read_stream_results`. The LLM plans; the tools execute against DWSIM.
8. Planner → Coder → Executor → Verifier loop:
   - Planner: decompose request into block diagram (units + streams + components).
   - Coder: emit DWSIM Python.
   - Executor: run in a sandboxed subprocess with DWSIM loaded.
   - Verifier: check convergence, mass/energy balance closure, no NaN streams. On failure, feed the error back.
9. Sandbox the generated code. LLM-generated Python touching a desktop app needs process isolation, a timeout, and a whitelisted import set.

Phase 4 — Iteration & domain feedback

10. Error taxonomy: catalog the failure modes (missing component, bad connection, non-converged recycle, wrong phase) and write targeted retry prompts for each. Generic "fix the error" loops waste tokens.

11. Evals: score on the reference flowsheets — does it produce a runnable simulation? Do key streams match expected compositions within tolerance? Run this on every prompt/model change.

Phase 5 — UX

12. Start with CLI, then add a UI that shows the generated code, the DWSIM result, and a diff when the user asks for revisions. Keep the human in the loop for the first version — chemical simulations have too many silent-failure modes for full autonomy.

Stack suggestion

- LLM: Claude (Sonnet 4.6 for the coder loop, Opus 4.7 for planning) via the Anthropic SDK with tool use + prompt caching on the API reference.
- Runtime: Python 3.11, `pythonnet`, DWSIM 8.x on Windows (or Linux via Mono — test early, this is the riskiest dependency).
- Orchestration: Claude Agent SDK or a thin custom loop — don't over-frameworkify until you've shipped a working prototype.

Biggest risk to address first: get a hand-written Python script to drive DWSIM end-to-end and produce a converged ammonia synthesis loop. If that takes a week, the rest of the project is tractable. If DWSIM Automation fights you, the whole plan needs to pivot to the IronPython path.

Here's a sketch for a simplified ammonia synthesis loop (fresh feed → mixer → compressor → reactor → cooler → flash → recycle). It's deliberately minimal but exercises every pattern the agent will later need to emit.

```
"""
Reference script: simplified ammonia synthesis loop in DWSIM via
Automation.
Goal: prove the API patterns end-to-end before any LLM gets involved.

NOTE: exact attribute names should be verified against your DWSIM version —
the Automation API has shifted between 7.x and 8.x. Treat this as a
structural template, not copy-paste-runnable.
"""

import clr, sys, os

# 1. Load DWSIM .NET assemblies via pythonnet
DWSIM_PATH = r"C:\Users\<you>\AppData\Local\DWSIM"
sys.path.append(DWSIM_PATH)
for asm in [
    "DWSIM.Automation", "DWSIM.Interfaces", "DWSIM.GlobalSettings",
    "DWSIM.SharedClasses", "DWSIM.Thermodynamics",
    "DWSIM.UnitOperations", "DWSIM.Inspector", "DWSIM.MathOps",
]:
    clr.AddReference(os.path.join(DWSIM_PATH, asm))

from DWSIM.Automation import Automation3
from DWSIM.Thermodynamics import Streams, PropertyPackages
from DWSIM.UnitOperations import UnitOperations
from DWSIM.Interfaces.Enums.GraphicObjects import ObjectType

# 2. Boot the automation host and create an empty flowsheet
auto = Automation3()
sim = auto.CreateFlowsheet()

# 3. Add compounds
for name in ["Nitrogen", "Hydrogen", "Ammonia"]:
    comp = sim.AvailableCompounds[name]
    sim.SelectedCompounds.Add(comp.Name, comp)

# 4. Attach a property package (Peng-Robinson is fine for this loop)
pp = PropertyPackages.PengRobinsonPropertyPackage()
pp.ComponentName = "PR"
sim.AddPropertyPackage(pp)

# 5. Helper to drop a unit op on the flowsheet canvas
def add(obj_type, tag, x, y):
    name = sim.AddObject(obj_type, x, y, tag)
    return sim.SimulationObjects[name]

feed     = add(ObjectType.MaterialStream,  "FreshFeed",  50,  100)
mixer    = add(ObjectType.NodeIn,          "Mixer",     150,  100)
mix_out  = add(ObjectType.MaterialStream,  "MixOut",    220,  100)
comp     = add(ObjectType.Compressor,      "Comp",      300,  100)
comp_out = add(ObjectType.MaterialStream,  "CompOut",   380,  100)
rxr      = add(ObjectType.RCT_Conversion,  "Reactor",   460,  100)
rxr_out  = add(ObjectType.MaterialStream,  "RxOut",     540,  100)
cooler   = add(ObjectType.Cooler,          "Cooler",    620,  100)
cool_out = add(ObjectType.MaterialStream,  "CoolOut",   700,  100)
flash    = add(ObjectType.Vessel,          "Flash",     780,  100)
product  = add(ObjectType.MaterialStream,  "NH3Prod",   860,  150)
```

```
recycle  = add(ObjectType.MaterialStream,  "Recycle",   860,   50)

# 6. Wire connections (output-port-of-A → input-port-of-B)
def connect(src, src_port, dst, dst_port):
    sim.ConnectObjects(src.GraphicObject, dst.GraphicObject, src_port,
dst_port)

connect(feed,     0, mixer,    0)
connect(recycle,  0, mixer,    1)
connect(mixer,    0, mix_out,  0)
connect(mix_out,  0, comp,     0)
connect(comp,     0, comp_out, 0)
connect(comp_out, 0, rxr,      0)
connect(rxr,      0, rxr_out,  0)
connect(rxr_out,  0, cooler,   0)
connect(cooler,   0, cool_out, 0)
connect(cool_out, 0, flash,    0)
connect(flash,    0, recycle,  0)   # vapor (mostly unreacted N2/H2)
connect(flash,    1, product,  0)   # liquid (ammonia)

# 7. Specify the fresh feed
feed.SetTemperature(298.15)               # K
feed.SetPressure(101325)                  # Pa
feed.SetMassFlow(1.0)                     # kg/s
feed.SetOverallComposition([0.25, 0.75, 0.0])  # N2, H2, NH3 (mole
fractions)

# 8. Unit-op parameters
comp.set_POut(20e6)                        # 200 bar discharge
rxr.set_OutletTemperature(723.15)          # 450 °C
# Reaction: N2 + 3H2 -> 2NH3, fractional conversion on N2
rxr.set_Conversion("N2", 0.25)
cooler.set_OutletTemperature(263.15)       # -10 °C to drop NH3 out
flash.set_Pressure(20e6)

# 9. Solve. For the recycle, DWSIM's solver iterates automatically once
#    a recycle block is inserted — this minimal example may need an
explicit
#    Recycle unit op (ObjectType.OT_Recycle) between flash-vapor and mixer
#    to converge cleanly. Add one if convergence fails.
sim.RequestCalculation()

# 10. Read back results — this is the shape the verifier needs
def stream_summary(s):
    return {
        "T_K":      s.GetTemperature(),
        "P_Pa":     s.GetPressure(),
        "mflow":    s.GetMassFlow(),
        "x":        list(s.GetOverallComposition()),
    }

print("Product:", stream_summary(product))
print("Recycle:", stream_summary(recycle))
print("Converged:", sim.Solved)
```

What this script is teaching you

- The seven primitives the LLM will eventually need to emit:

  load assemblies → create sim → add compounds → set property package → add objects → connect → specify → solve → read.

  Every generated script is a permutation of these.

- The connection model: by port index, against `GraphicObject`s. This is the most common place LLM-generated code breaks, so the tool layer should expose `connect(src_tag, dst_tag)` and resolve ports internally.
- The result shape (`stream_summary`): this is exactly what your Verifier consumes to check mass balance and convergence.

What to nail down before moving on

1. Run this against your DWSIM install and fix the attribute names that don't match — the Automation API uses different setter conventions across versions (`SetTemperature` vs `Temperature = ...` vs `set_Temperature`). Lock the version.
2. Decide on the Recycle block: real recycle loops need DWSIM's `OT_Recycle` unit op with a tear stream, not just a stream that loops back. Get one working here so the agent has a template.
3. Wrap the patterns into a thin Python module (`dwsim_kit.py`) exposing `add`, `connect`, `stream_summary`, etc. Then the LLM emits code against `dwsim_kit`, not raw DWSIM — far smaller surface area, far fewer hallucinations.

Want me to sketch `dwsim_kit.py` (the LLM-facing wrapper) or the tool schema for the agent next?

Here's `dwsim_kit.py`. The idea: the LLM never imports DWSIM, never touches GraphicObjects, never juggles port indices. It writes short declarative Python against this kit, and the kit validates everything before DWSIM ever runs.

```
"""
dwsim_kit.py — LLM-facing wrapper around DWSIM Automation.

Design goals:
  1. Tag-based addressing. No GraphicObjects, no port indices.
  2. Friendly units at the boundary (bar/°C/kg-per-h); SI internally.
  3. Loud, actionable errors before any DWSIM call.
  4. Small enumerable catalog the agent can introspect at code-gen time.
  5. Results returned as plain dicts so the Verifier can score them.
"""

from __future__ import annotations
from dataclasses import dataclass, field
from typing import Optional
import math

from . import _dwsim_backend as backend  # wraps the pythonnet calls

# ---------------------------------------------------------------------------
# Unit conversions (applied only at the kit boundary)
# ---------------------------------------------------------------------------

def _to_K(T, unit="C"):
    if unit == "K": return T
    if unit == "C": return T + 273.15
    if unit == "F": return (T - 32) * 5/9 + 273.15
    raise KitError(f"unknown temperature unit {unit!r}")

def _to_Pa(P, unit="bar"):
    f = {"Pa": 1, "kPa": 1e3, "bar": 1e5, "MPa": 1e6, "atm": 101325}
    if unit not in f: raise KitError(f"unknown pressure unit {unit!r}")
    return P * f[unit]

def _to_kg_s(m, unit="kg/h"):
    if unit == "kg/s": return m
    if unit == "kg/h": return m / 3600
    if unit == "t/h":  return m * 1000 / 3600
    raise KitError(f"unknown mass-flow unit {unit!r}")

# ---------------------------------------------------------------------------
# Catalog — the agent introspects this to know what it can use
# ---------------------------------------------------------------------------

UNIT_CATALOG = {
    "Mixer":           {"in": "many", "out": 1},
    "Splitter":        {"in": 1, "out": "many", "required":
["split_ratios"]},
    "Compressor":      {"in": 1, "out": 1, "required": ["P_out"],
                        "optional": ["efficiency"]},
    "Pump":            {"in": 1, "out": 1, "required": ["P_out"]},
    "Heater":          {"in": 1, "out": 1, "one_of": ["T_out", "duty"]},
    "Cooler":          {"in": 1, "out": 1, "one_of": ["T_out", "duty"]},
    "HeatExchanger":   {"in": 2, "out": 2, "required": ["UA"]},
```

```
    "ConversionReactor": {"in": 1, "out": 1,
                          "required": ["reactions", "T_out"]},
    "GibbsReactor":      {"in": 1, "out": 1, "required": ["T_out",
"P_out"]},
    "Flash":             {"in": 1, "out": 2,           # vapor, liquid
                          "ports_out": ["vapor", "liquid"],
                          "required": ["P", "T"]},
    "DistillationColumn":{"in": "many", "out": "many",
                          "ports_out": ["distillate", "bottoms"],
                          "required": ["n_stages", "feed_stage",
                                       "reflux_ratio", "boilup_ratio"]},
    "Valve":             {"in": 1, "out": 1, "required": ["P_out"]},
}

PROPERTY_PACKAGES = {
    "PengRobinson", "SRK", "NRTL", "UNIQUAC", "Raoult", "SteamTables",
}

# ------------------------------------------------------------------------
--
# Pre-solve records (kit-side only; never leak into LLM-visible API)
# ------------------------------------------------------------------------
--

@dataclass
class StreamSpec:
    tag: str
    T_K: Optional[float] = None
    P_Pa: Optional[float] = None
    mass_flow_kg_s: Optional[float] = None
    composition: dict[str, float] = field(default_factory=dict)  # mole
fractions

@dataclass
class UnitSpec:
    tag: str
    kind: str
    params: dict = field(default_factory=dict)

@dataclass
class Connection:
    src: str   # "tag" or "tag.port_name" (e.g. "Flash1.vapor")
    dst: str

class KitError(ValueError):
    """Raised before any DWSIM call. Always actionable."""

# ------------------------------------------------------------------------
--
# The Flowsheet — the only class the LLM uses
# ------------------------------------------------------------------------
--

class Flowsheet:
    def __init__(self, name: str = "flowsheet"):
        self.name = name
        self.compounds: list[str] = []
        self.pp: Optional[str] = None
        self.streams: dict[str, StreamSpec] = {}
        self.units:   dict[str, UnitSpec]   = {}
        self.connections: list[Connection] = []
```

```
        self.recycles:    list[Connection] = []

    # -- declarative setup --------------------------------------------
--

    def use_compounds(self, compounds: list[str]) -> None:
        unknown = [c for c in compounds if c not in
backend.AVAILABLE_COMPOUNDS]
        if unknown:
            raise KitError(f"unknown compounds: {unknown}. "
                           f"Call Flowsheet.list_compounds() to see the
catalog.")
        self.compounds = compounds

    def use_property_package(self, pp: str) -> None:
        if pp not in PROPERTY_PACKAGES:
            raise KitError(f"unknown property package {pp!r}. "
                           f"Choose from {sorted(PROPERTY_PACKAGES)}.")
        self.pp = pp

    def add_stream(self, tag, *, T=None, T_unit="C",
                   P=None, P_unit="bar",
                   mass_flow=None, mass_flow_unit="kg/h",
                   composition: Optional[dict[str, float]] = None) -> None:
        self._check_tag_unused(tag)
        if composition: self._check_composition(composition)
        self.streams[tag] = StreamSpec(
            tag=tag,
            T_K  = _to_K(T,  T_unit)  if T  is not None else None,
            P_Pa = _to_Pa(P, P_unit)  if P  is not None else None,
            mass_flow_kg_s = _to_kg_s(mass_flow, mass_flow_unit)
                             if mass_flow is not None else None,
            composition = composition or {},
        )

    def add_unit(self, kind: str, tag: str, **params) -> None:
        if kind not in UNIT_CATALOG:
            raise KitError(f"unknown unit type {kind!r}. "
                           f"Catalog: {sorted(UNIT_CATALOG)}.")
        self._check_tag_unused(tag)
        self._check_unit_params(kind, params)
        self.units[tag] = UnitSpec(tag=tag, kind=kind, params=params)

    def connect(self, src: str, dst: str) -> None:
        self._check_endpoint(src)
        self._check_endpoint(dst)
        self.connections.append(Connection(src=src, dst=dst))

    def add_recycle(self, *, from_: str, to: str) -> None:
        """Mark a recycle loop. The backend inserts a tear/Recycle unit
op."""
        self._check_endpoint(from_)
        self._check_endpoint(to)
        self.recycles.append(Connection(src=from_, dst=to))

    # -- validation + solve -------------------------------------------
--

    def validate(self) -> list[str]:
        """Pure-Python checks. The agent runs this before solve() and feeds
        any issues back to the LLM for self-repair."""
```

```
        issues = []
        if not self.compounds: issues.append("no compounds declared")
        if not self.pp:        issues.append("no property package
declared")

        dsts = {c.dst.split(".")[0] for c in self.connections}
        feeds = [t for t in self.streams if t not in dsts]
        for tag in feeds:
            s = self.streams[tag]
            for f in ("T_K", "P_Pa", "mass_flow_kg_s"):
                if getattr(s, f) is None:
                    issues.append(f"feed stream {tag!r} missing {f}")
            if not s.composition:
                issues.append(f"feed stream {tag!r} missing composition")

        for u in self.units.values():
            if not any(c.dst.split(".")[0] == u.tag for c in
self.connections):
                issues.append(f"unit {u.tag!r} has no inbound connection")
            if not any(c.src.split(".")[0] == u.tag for c in
self.connections):
                issues.append(f"unit {u.tag!r} has no outbound connection")
        return issues

    def solve(self, timeout_s: int = 120) -> "SolveResult":
        issues = self.validate()
        if issues:
            raise KitError("flowsheet invalid:\n  - " + "\n  -
".join(issues))
        handle = backend.build_and_solve(
            name=self.name,
            compounds=self.compounds,
            property_package=self.pp,
            streams=self.streams,
            units=self.units,
            connections=self.connections,
            recycles=self.recycles,
            timeout_s=timeout_s,
        )
        return SolveResult(handle)

    # -- introspection (also exposed as agent tools) ----------------------
--

    @staticmethod
    def list_unit_types() -> dict:        return UNIT_CATALOG
    @staticmethod
    def list_property_packages() -> set:  return PROPERTY_PACKAGES
    @staticmethod
    def list_compounds() -> list[str]:    return
backend.AVAILABLE_COMPOUNDS

    # -- private helpers --------------------------------------------------
--

    def _check_tag_unused(self, tag):
        if tag in self.streams or tag in self.units:
            raise KitError(f"tag {tag!r} already used")

    def _check_composition(self, comp):
        unknown = [c for c in comp if c not in self.compounds]
```

```
        if unknown:
            raise KitError(f"composition refers to undeclared compounds:
{unknown}")
        s = sum(comp.values())
        if not math.isclose(s, 1.0, abs_tol=1e-3):
            raise KitError(f"composition fractions sum to {s:.4f}, not
1.0")

    def _check_unit_params(self, kind, params):
        spec = UNIT_CATALOG[kind]
        for req in spec.get("required", []):
            if req not in params:
                raise KitError(f"{kind} requires parameter {req!r}")
        one_of = spec.get("one_of")
        if one_of and not any(k in params for k in one_of):
            raise KitError(f"{kind} requires one of {one_of}")

    def _check_endpoint(self, ref):
        tag, *port = ref.split(".")
        if tag not in self.streams and tag not in self.units:
            raise KitError(f"unknown endpoint {ref!r}")
        if port:
            spec = UNIT_CATALOG.get(self.units[tag].kind, {}) if tag in
self.units else {}
            valid = spec.get("ports_out", [])
            if port[0] not in valid:
                raise KitError(f"{tag!r} has no port {port[0]!r}; valid:
{valid}")


# ---------------------------------------------------------------------------
--
# Results — shape consumed by the Verifier
# ---------------------------------------------------------------------------
--

class SolveResult:
    def __init__(self, handle): self._h = handle
    @property
    def converged(self) -> bool:                return self._h.solved
    def stream(self, tag: str) -> dict:         return
backend.stream_summary(self._h, tag)
    def all_streams(self) -> dict[str, dict]:   return
backend.all_streams(self._h)
    def mass_balance(self) -> dict:             return
backend.mass_balance(self._h)
    def energy_balance(self) -> dict:           return
backend.energy_balance(self._h)
```

What the LLM-emitted code now looks like

Same ammonia loop, but this is what gets generated and executed:

```
from dwsim_kit import Flowsheet

sim = Flowsheet("ammonia_loop")
sim.use_compounds(["Nitrogen", "Hydrogen", "Ammonia"])
sim.use_property_package("PengRobinson")

sim.add_stream("FreshFeed",
               T=25, P=1, mass_flow=3600,           # 1 kg/s
               composition={"Nitrogen": 0.25, "Hydrogen": 0.75})
```

```
sim.add_unit("Mixer",              "M1")
sim.add_unit("Compressor",         "C1", P_out=200)              # bar
sim.add_unit("ConversionReactor", "R1",
             T_out=450,
             reactions=[{"stoich": {"Nitrogen": -1, "Hydrogen": -3,
"Ammonia": 2},
                         "basis": "Nitrogen", "conversion": 0.25}])
sim.add_unit("Cooler",             "E1", T_out=-10)
sim.add_unit("Flash",              "V1", T=-10, P=200)

sim.connect("FreshFeed",  "M1")
sim.connect("M1",         "C1")
sim.connect("C1",         "R1")
sim.connect("R1",         "E1")
sim.connect("E1",         "V1")

sim.add_recycle(from_="V1.vapor", to="M1")
# V1.liquid is implicitly the product stream

result = sim.solve()
print("converged:", result.converged)
print("product:",   result.stream("V1.liquid"))
print("balance:",   result.mass_balance())
```

That's the entire program. Compare to the raw DWSIM script from the previous phase — same simulation, ~1/3 the lines, zero references to ports or graphic objects, and every error surfaces as a sentence the LLM can act on.

Why this shape pays off

- Smaller hallucination surface. The LLM only needs to know ~12 unit types and ~6 property packages. It can call `Flowsheet.list_unit_types()` as a tool at code-gen time to ground itself.
- Validation is free fuel for the agent loop. `validate()` returns a `list[str]`; on failure you concatenate those into the next LLM prompt verbatim. No need to parse DWSIM tracebacks.
- Unit safety. The LLM thinks in bar and °C like a chemical engineer would. SI conversion is the kit's problem.
- Result shape is stable. `result.stream(tag)` always returns `{T_K, P_Pa, mass_flow, composition, phase, ...}` — the Verifier can write deterministic mass-balance checks without caring how DWSIM laid out its objects.

Build order for this file

1. Stub `_dwsim_backend` first — return fake `SolveResult`s — and write unit tests against the kit's validation paths. This decouples kit design from DWSIM debugging.
2. Wire the backend using the patterns from the Phase 1 reference script.
3. Add one unit op at a time to the catalog, with a smoke test that builds-and-solves the simplest flowsheet using only that op.
4. Only then start prompting an LLM to emit `dwsim_kit` code.

Want me to sketch the agent tool schema (the function-calling surface the LLM sees) or the Verifier next?

Here's the tool surface the LLM sees, plus the system-prompt skeleton that binds it together.

Design principles

1. One script per attempt, not 50 tool calls. The LLM emits a complete `dwsim_kit` program and calls `run_flowsheet` once. Each round-trip is a planning checkpoint, not a single API call.
2. Knowledge tools are read-only and cheap. The LLM should hit them freely during planning to ground itself in real compound names, real unit ops, real reference topologies.
3. Errors are tools, not exceptions. When a run fails, the LLM doesn't see a traceback — it sees structured diagnostics shaped for self-repair.
4. No raw Python import. `run_flowsheet` accepts a script string but the executor only exposes `dwsim_kit`. The LLM can't `import os` and escape.

The schemas (Anthropic tool-use format)

```
TOOLS = [

  # ── 1. KNOWLEDGE: planning phase ──────────────────────────────────

  {
    "name": "list_unit_operations",
    "description": (
      "List every unit operation type available in dwsim_kit, with its "
      "required and optional parameters. Call this FIRST when planning a "
      "flowsheet to ground your design in what the kit actually supports."
    ),
    "input_schema": {"type": "object", "properties": {}, "required": []},
  },

  {
    "name": "get_unit_operation_details",
    "description": (
      "Return full details for one unit op: port topology, every parameter
"
      "with units and valid ranges, and a minimal worked example."
    ),
    "input_schema": {
      "type": "object",
      "properties": {
        "kind": {"type": "string",
                 "description": "Unit op name, e.g. 'ConversionReactor'."}
      },
      "required": ["kind"],
    },
  },

  {
    "name": "find_compounds",
    "description": (
      "Search DWSIM's compound database. DWSIM is strict about spelling — "
      "'NH3' will NOT match 'Ammonia'. Use this to resolve common names, "
      "formulas, or CAS numbers into the exact names dwsim_kit expects."
    ),
    "input_schema": {
      "type": "object",
      "properties": {
        "query": {"type": "string",
```

```
                    "description": "Common name, formula, or CAS number."},
          "limit": {"type": "integer", "default": 10},
        },
        "required": ["query"],
      },
    },

    {
      "name": "list_property_packages",
      "description": "List thermodynamic property packages dwsim_kit
supports.",
      "input_schema": {"type": "object", "properties": {}, "required": []},
    },

    {
      "name": "recommend_property_package",
      "description": (
        "Given a compound set and a rough operating regime, recommend a "
        "thermodynamic package and explain why. Use BEFORE finalising the "
        "script — picking a wrong package is the #1 cause of bad results "
        "that still 'converge'."
      ),
      "input_schema": {
        "type": "object",
        "properties": {
          "compounds": {"type": "array", "items": {"type": "string"}},
          "regime":    {"type": "string",
                        "enum": ["low_P_polar", "high_P_nonpolar",
                                 "aqueous_electrolyte", "steam", "general"]},
        },
        "required": ["compounds", "regime"],
      },
    },

    {
      "name": "find_reference_flowsheet",
      "description": (
        "Retrieve a canonical block diagram for a named process (e.g. "
        "'ammonia synthesis', 'methanol loop', 'crude distillation'). "
        "Returns a topology sketch — units, streams, key parameters — that "
        "you can adapt rather than inventing from scratch."
      ),
      "input_schema": {
        "type": "object",
        "properties": {
          "process": {"type": "string"},
        },
        "required": ["process"],
      },
    },

    # ── 2. ACTION: build + execute ──────────────────────────────────────

    {
      "name": "run_flowsheet",
      "description": (
        "Execute a dwsim_kit script in the sandboxed DWSIM runtime. The "
        "script may import ONLY from `dwsim_kit`. Returns either a success "
        "envelope (with stream results, mass/energy balance, run_id) or a "
        "failure envelope with structured diagnostics — never a raw "
```

```
      "traceback. On failure, read `diagnostics` carefully before
retrying."
    ),
    "input_schema": {
      "type": "object",
      "properties": {
        "script": {
          "type": "string",
          "description": "Complete Python program using dwsim_kit.",
        },
        "label": {
          "type": "string",
          "description": "Short human-readable name for this attempt, e.g.
"
                         "'ammonia_loop_v1'. Shown to the user.",
        },
      },
      "required": ["script", "label"],
    },
  },

  # ── 3. INSPECTION: post-run ──────────────────────────────────────

  {
    "name": "inspect_streams",
    "description": (
      "Query one or more streams from a successful run. Cheaper than "
      "re-running. Returns T, P, mass flow, composition, phase fractions."
    ),
    "input_schema": {
      "type": "object",
      "properties": {
        "run_id":  {"type": "string"},
        "streams": {"type": "array", "items": {"type": "string"},
                    "description": "Stream tags, e.g. ['V1.liquid',
'M1.out']"},
      },
      "required": ["run_id", "streams"],
    },
  },

  {
    "name": "solver_diagnostics",
    "description": (
      "When `run_flowsheet` returns a non-convergence or solver-error "
      "envelope, call this for deeper signals: which unit failed first, "
      "recycle tear-stream history, last residuals, suspected cause."
    ),
    "input_schema": {
      "type": "object",
      "properties": {"run_id": {"type": "string"}},
      "required": ["run_id"],
    },
  },

  # ── 4. HANDOFF ──────────────────────────────────────────────────

  {
    "name": "report_to_user",
    "description": (
      "Final answer for the user. Summarises what was built, key stream "
```

```
      "conditions, mass-balance closure, and points at the saved .dwxmz "
      "file path. Call this exactly once at the end of a successful task."
    ),
    "input_schema": {
      "type": "object",
      "properties": {
        "run_id":          {"type": "string"},
        "summary":         {"type": "string"},
        "key_streams":     {"type": "array", "items": {"type": "string"}},
        "assumptions":     {"type": "array", "items": {"type": "string"},
                            "description": "Anything you had to assume "
                                           "because the user didn't
specify."},
        "open_questions":  {"type": "array", "items": {"type": "string"}},
      },
      "required": ["run_id", "summary", "key_streams"],
    },
  },
]
```

Response envelope from `run_flowsheet`

This is the half people skip — but it determines whether the agent loop actually converges. Two shapes, both deterministic:

```
# success
{
  "status": "ok",
  "run_id": "r_8af3",
  "converged": True,
  "streams": {
    "FreshFeed":  {"T_K": 298.15, "P_Pa": 1.01e5, "mass_flow_kg_s": 1.0,
                   "composition": {"Nitrogen": 0.25, ...}, "phase":
"vapor"},
    "V1.liquid":  {...},
    ...
  },
  "mass_balance":   {"in_kg_s": 1.0, "out_kg_s": 0.998, "closure_pct":
99.8},
  "energy_balance": {...},
  "dwxmz_path": "/runs/r_8af3/ammonia_loop_v1.dwxmz",
}

# failure — every field is for the LLM to act on
{
  "status": "error",
  "run_id": "r_8af3",
  "error_class": "validation" | "build" | "solve" | "timeout",
  "issues": [
    "feed stream 'FreshFeed' missing P_Pa",
    "unit 'R1' references undeclared compound 'NH3' "
      "(did you mean 'Ammonia'? call find_compounds)",
  ],
  "failing_unit": "R1",
  "hint": "ConversionReactor requires reactant names that match "
          "compounds declared via use_compounds().",
}
```

The `hint` field is the one that disproportionately determines success. Curate hints by failure class — generic ones waste tokens, specific ones close the loop in one retry.

System prompt skeleton

```
You are a chemical process engineer building DWSIM flowsheets.
Workflow:
  1. PLAN. Use find_reference_flowsheet for the named process. Use
     find_compounds to resolve every chemical name. Use
     recommend_property_package given the compound set and regime.
     Use list_unit_operations / get_unit_operation_details until you
     know exactly which ops you'll use.
  2. EMIT. Write a complete dwsim_kit script in one shot. Imports
     from dwsim_kit only. No comments. Include every required parameter.
  3. RUN. Call run_flowsheet with the script. Read the envelope.
  4. REPAIR. On error, read `issues` and `hint`. Fix the SPECIFIC
     problem named — do not rewrite the whole script. Then run again.
     Stop after 4 failed attempts and call report_to_user with the
     blocker spelled out.
  5. VERIFY. On success, check mass_balance.closure_pct > 99.5.
     Use inspect_streams to confirm key product purities.
  6. REPORT. Call report_to_user exactly once.
Hard rules:
  - Never invent unit ops or property packages. Stick to the catalog.
  - Never guess a compound name. Always find_compounds first.
  - If the user under-specifies, make ONE reasonable assumption per
     missing input and record it in `assumptions` at report time.
  - Recycles must be closed with add_recycle(from_=, to=). Do not
     create a loop with plain connect() calls.
```

The loop in pseudocode

```
messages = [{"role": "user", "content": user_request}]
for step in range(MAX_STEPS):
    resp = anthropic.messages.create(
        model="claude-opus-4-7",
        system=SYSTEM_PROMPT,
        tools=TOOLS,
        messages=messages,
        # cache the catalog tools — they don't change between turns
        extra_headers={"anthropic-beta": "prompt-caching-2024-07-31"},
    )
    messages.append({"role": "assistant", "content": resp.content})
    if resp.stop_reason == "end_turn":
        break
    tool_results = [execute(tu) for tu in tool_uses(resp)]
    messages.append({"role": "user", "content": tool_results})
    if any(tu.name == "report_to_user" for tu in tool_uses(resp)):
        break
```

What to nail before building this

1. Author the `find_reference_flowsheet` corpus. 10–15 hand-written templates is enough; this is the single biggest lever on first-attempt success rate. The LLM is bad at inventing unit topology and great at adapting examples.
2. Curate the `hint` table. Map each `error_class × failing_unit_kind` combination to one specific repair instruction. Generic hints leak tokens; specific ones close the agent loop in one shot.

3. Decide your `MAX_STEPS` budget. Start at 8 — enough for plan (3) + build (1) + repair (3) + report (1). If you're routinely hitting the cap, that's a signal to improve hints, not raise the cap.
4. Cache the catalog tools' responses. `list_unit_operations` and `list_property_packages` return the same payload every call — wrap them with prompt caching so repeated planning turns stay cheap.

Want me to sketch the Verifier next (the mass-balance / convergence-checking layer that consumes `run_flowsheet`'s success envelope), or the `find_reference_flowsheet` corpus format (what one canonical template entry looks like)?

Here's the Verifier. Core insight first: **convergence is not correctness.** DWSIM will gladly return `converged=True` with mass balance off by 30% (recycle solver bailed at max iterations), broken element balance (typo in reaction stoichiometry — atoms shuffled but mass stays closed), or the product stream in the wrong phase. The Verifier's job is to catch all of that deterministically and hand the LLM a repair-shaped report.

```
"""
verifier.py — sanity-check a converged DWSIM run.

Runs automatically after every successful run_flowsheet. The result is
appended to the envelope as `verification`, so the LLM always sees it.
"""

from __future__ import annotations
from dataclasses import dataclass, field
from typing import Optional, Literal

Severity = Literal["error", "warning", "info"]

# Tolerances — strict by default. Relax only with a recorded justification.
MASS_CLOSURE_PCT    = 99.5
ELEM_CLOSURE_PCT    = 99.0
ENERGY_CLOSURE_PCT  = 95.0    # heat losses make 99.5 noisy in practice
RECYCLE_RESIDUAL    = 1e-3
TRIVIAL_FLOW_KG_S   = 1e-9


@dataclass
class Finding:
    check: str
    severity: Severity
    message: str
    detail: dict = field(default_factory=dict)
    repair_hint: Optional[str] = None       # the field the agent loop
reads


@dataclass
class VerificationReport:
    findings: list[Finding] = field(default_factory=list)

    @property
    def has_errors(self) -> bool:
        return any(f.severity == "error" for f in self.findings)

    @property
    def errors(self):   return [f for f in self.findings if f.severity ==
"error"]
    @property
    def warnings(self): return [f for f in self.findings if f.severity ==
"warning"]

    def add(self, **kw): self.findings.append(Finding(**kw))

    def as_llm_message(self) -> str:
        if not self.findings:
            return "Verification passed."
        out = []
        for f in self.errors + self.warnings:
            tag = "ERROR" if f.severity == "error" else "WARN"
```

```
            out.append(f"[{tag}] {f.check}: {f.message}")
            if f.repair_hint:
                out.append(f"  → {f.repair_hint}")
        return "\n".join(out)


# ------------------------------------------------------------------------
--
# Intent — declared by the LLM before run_flowsheet, used to score the run
# ------------------------------------------------------------------------
--

@dataclass
class Intent:
    feed_streams:    list[str]
    product_streams: list[str]
    targets: dict = field(default_factory=dict)
    # Examples:
    #   "unit_setpoint":  ("R1", "T_out_K", 723.15, 5.0)   # value,
tolerance
    #   "product_purity": ("V1.liquid", "Ammonia", 0.95)
    #   "min_yield":      ("Ammonia", 0.20)                # mol/s in
products
    elements: Optional[dict[str, dict[str, int]]] = None
    # {"Ammonia": {"N": 1, "H": 3}, ...} — auto-filled by the kit


# ------------------------------------------------------------------------
--
# Checks. Each one is small, deterministic, and emits at most a few
findings.
# ------------------------------------------------------------------------
--

def check_convergence(env, report):
    if not env.get("converged", False):
        report.add(check="convergence", severity="error",
            message="DWSIM did not report convergence.",
            repair_hint="Call solver_diagnostics. If a recycle is open, "
                        "add an explicit Recycle unit op on the tear
stream.")

def check_trivial(env, report):
    feeds = [s for s in env["streams"].values() if s.get("is_feed")]
    if feeds and all(s["mass_flow_kg_s"] < TRIVIAL_FLOW_KG_S for s in
feeds):
        report.add(check="trivial_solution", severity="error",
            message="All feed flows are ~zero; solver returned the trivial
solution.",
            repair_hint="Confirm every fresh-feed stream has mass_flow
set.")

def check_mass_balance(env, report):
    mb = env.get("mass_balance") or {}
    closure = mb.get("closure_pct")
    if closure is None or closure >= MASS_CLOSURE_PCT: return
    delta = mb["in_kg_s"] - mb["out_kg_s"]
    report.add(check="mass_balance",
        severity="error" if closure < 95 else "warning",
        message=f"Mass balance closes at {closure:.2f}% (Δ={delta:.4g}
kg/s).",
```

```
        detail=mb,
        repair_hint="Usually an unconverged recycle (raise solver
iterations, "
                    "or check the tear-stream initial guess), or a Splitter
"
                    "whose ratios don't sum to 1.")

def check_element_balance(env, intent, report):
    """Atom balance catches reaction-stoichiometry typos that mass balance
    misses — atoms swapped between compounds can keep mass closed while
    breaking elements."""
    if not intent.elements: return
    in_atoms  = _atom_totals(env, intent, feeds=True)
    out_atoms = _atom_totals(env, intent, feeds=False)
    for el, ai in in_atoms.items():
        if ai < 1e-9: continue
        ao = out_atoms.get(el, 0.0)
        closure = 100 * ao / ai
        if closure < ELEM_CLOSURE_PCT:
            report.add(check=f"element_balance.{el}", severity="error",
                message=f"{el} balance closes at {closure:.2f}% "
                        f"(in={ai:.4g}, out={ao:.4g} mol/s).",
                detail={"element": el, "in": ai, "out": ao},
                repair_hint=f"At least one reaction is unbalanced for {el}.
"
                            f"Re-check stoichiometric coefficients.")

def check_unphysical(env, report):
    for tag, s in env["streams"].items():
        if s.get("T_K", 1) <= 0:
            report.add(check=f"unphysical.{tag}.T", severity="error",
                message=f"{tag} has T ≤ 0 K.")
        for c, x in (s.get("composition") or {}).items():
            if x < -1e-6:
                report.add(check=f"unphysical.{tag}.x", severity="error",
                    message=f"{tag} has negative {c} fraction {x:.3e}.")
        x_sum = sum((s.get("composition") or {}).values())
        if s.get("mass_flow_kg_s", 0) > TRIVIAL_FLOW_KG_S \
           and abs(x_sum - 1.0) > 1e-2:
            report.add(check=f"unphysical.{tag}.x_sum", severity="warning",
                message=f"{tag} composition sums to {x_sum:.4f}, not 1.0.")

def check_recycle_residuals(env, report):
    for r in env.get("recycles", []):
        if (r.get("residual") or 0) > RECYCLE_RESIDUAL:
            report.add(check=f"recycle.{r['tag']}", severity="warning",
                message=f"Recycle {r['tag']!r} residual {r['residual']:.2e}
"
                        f"> {RECYCLE_RESIDUAL:.0e}.",
                repair_hint="Raise solver max iterations, or check whether
"
                            "an upstream spec is fighting the loop.")

def check_setpoints(env, intent, report):
    for kind, payload in intent.targets.items():
        if kind != "unit_setpoint": continue
        tag, prop, expected, tol = payload
        actual = (env.get("units") or {}).get(tag, {}).get(prop)
        if actual is None:
            report.add(check=f"setpoint.{tag}.{prop}", severity="warning",
                message=f"Couldn't read actual {prop} for {tag}.")
```

```
            continue
        if abs(actual - expected) > tol:
            report.add(check=f"setpoint.{tag}.{prop}", severity="error",
                message=f"{tag}.{prop} = {actual:.4g}, requested
{expected:.4g} "
                        f"(tol ±{tol}).",
                repair_hint=f"{tag} may be over- or under-specified. Look
for "
                            f"another constraint forcing {prop} off-
setpoint.")

def check_product_specs(env, intent, report):
    for kind, payload in intent.targets.items():
        if kind == "product_purity":
            tag, compound, threshold = payload
            x = (env["streams"].get(tag, {}).get("composition") or
{}).get(compound, 0.0)
            if x < threshold:
                report.add(check=f"purity.{tag}.{compound}",
severity="warning",
                    message=f"{compound} purity in {tag} is {x:.3f}, "
                            f"target ≥ {threshold:.3f}.",
                    repair_hint="Tighten separation: lower flash T, raise "
                                "column reflux, or add a polishing stage.")
        elif kind == "min_yield":
            compound, threshold = payload
            y = _product_molar_flow(env, intent, compound)
            if y < threshold:
                report.add(check=f"yield.{compound}", severity="warning",
                    message=f"{compound} yield {y:.4g} mol/s <
{threshold:.4g}.",
                    repair_hint="Increase reactor conversion (check
approach "
                                "to equilibrium) or recycle ratio.")

def check_phases(env, intent, report):
    for tag in intent.product_streams:
        s = env["streams"].get(tag) or {}
        port = tag.split(".")[-1] if "." in tag else None
        if port == "liquid" and s.get("phase") != "liquid":
            report.add(check=f"phase.{tag}", severity="warning",
                message=f"{tag} expected liquid but is {s.get('phase')}.",
                repair_hint="Drop separator T or raise P; confirm the
stream "
                            "is below its bubble point.")
        if port == "vapor" and s.get("phase") not in ("vapor", "gas"):
            report.add(check=f"phase.{tag}", severity="warning",
                message=f"{tag} expected vapor but is {s.get('phase')}.")


# ----------------------------------------------------------------------
--
# Orchestration
# ----------------------------------------------------------------------
--

CHECKS_ALWAYS  = [check_convergence, check_trivial, check_unphysical,
                  check_mass_balance, check_recycle_residuals]
CHECKS_INTENT  = [check_element_balance, check_setpoints,
                  check_product_specs, check_phases]
```

```
def verify(envelope: dict, intent: Intent) -> VerificationReport:
    report = VerificationReport()
    for c in CHECKS_ALWAYS:  c(envelope, report)
    for c in CHECKS_INTENT:  c(envelope, intent, report)
    return report


# ------------------------------------------------------------------------
--
# Helpers
# ------------------------------------------------------------------------
--

def _atom_totals(env, intent, feeds: bool):
    totals = {}
    for tag, s in env["streams"].items():
        if bool(s.get("is_feed")) != feeds: continue
        n_total = s.get("molar_flow_mol_s", 0)
        for c, x in (s.get("composition") or {}).items():
            atoms = intent.elements.get(c, {})
            for el, count in atoms.items():
                totals[el] = totals.get(el, 0) + n_total * x * count
    return totals

def _product_molar_flow(env, intent, compound):
    total = 0
    for tag in intent.product_streams:
        s = env["streams"].get(tag) or {}
        x = (s.get("composition") or {}).get(compound, 0)
        total += s.get("molar_flow_mol_s", 0) * x
    return total
```

How it plugs into the agent loop

The Verifier is **not** an LLM tool. It runs inside the executor, after every successful run, and its output is grafted onto the envelope before the LLM sees the response:

```
def run_flowsheet_tool(script: str, label: str) -> dict:
    intent = parse_intent_from_session()    # last declare_intent call
    envelope = execute_in_sandbox(script, label)
    if envelope["status"] == "ok":
        report = verify(envelope, intent)
        envelope["verification"] = {
            "passed":   not report.has_errors,
            "summary":  report.as_llm_message(),
            "findings": [f.__dict__ for f in report.findings],
        }
    return envelope
```

So what the LLM gets back, even on a "successful" run, looks like:

```
{
  "status": "ok",
  "converged": true,
  "run_id": "r_8af3",
  "streams": { ... },
  "mass_balance": { "closure_pct": 96.4, ... },
  "verification": {
```

```
    "passed": false,
    "summary": "[ERROR] mass_balance: closes at 96.40% (Δ=0.036 kg/s).\n  →
Usually an unconverged recycle...\n[WARN] purity.V1.liquid.Ammonia: is
0.812, target ≥ 0.950.\n  → Tighten separation...",
    "findings": [ ... ]
  }
}
```

The agent's stop condition becomes simply: `converged && verification.passed`. Warnings show up in `report_to_user` but don't block. Errors loop.

One extra tool the LLM needs

To make this work, add a `declare_intent` tool to the schema from the last step. The LLM calls it once after planning, before `run_flowsheet`:

```
{
  "name": "declare_intent",
  "description": (
    "Declare what this flowsheet is supposed to achieve. The Verifier "
    "uses this to score correctness beyond mere convergence. Call once "
    "after planning, before the first run_flowsheet."
  ),
  "input_schema": {
    "type": "object",
    "properties": {
      "feed_streams":    {"type": "array", "items": {"type": "string"}},
      "product_streams": {"type": "array", "items": {"type": "string"}},
      "targets":         {"type": "object",
                          "description": "Mix of unit_setpoint, "
                                         "product_purity, min_yield
entries."},
    },
    "required": ["feed_streams", "product_streams"],
  },
}
```

What to nail before shipping

1. **Decide error vs. warning per check.** Errors must loop the agent; warnings can ship with the report. Get this wrong in either direction and you either fail-loop on cosmetic issues or ship broken sims silently. Start with my defaults and revise from eval data.
2. **Tune the tolerances on your eval set, not in your head.** Run the 10–15 reference flowsheets through the Verifier and pick thresholds that pass the good ones and fail the seeded-bad ones. Don't tune on user traffic.
3. **The element-balance check is your highest-leverage one.** It catches the failure mode the LLM is most prone to (stoichiometry typos in `ConversionReactor`). Make sure `intent.elements` is auto-populated from the kit's compound DB — never trust the LLM to declare it.
4. **`repair_hint` is the product.** Every error class needs a hint that, fed back to the LLM, fixes the run in one retry. Treat hint quality as the eval metric for the Verifier — measure "errors closed per retry" on your eval set and iterate on hints, not on adding more checks.

Want me to sketch the executor sandbox next (how `run_flowsheet` actually runs the LLM-generated script safely), or the `find_reference_flowsheet` corpus entry format?

Here's the sandbox. Threat model first — keep this in mind throughout: **the script is LLM output, so treat it as untrusted code.** Not because the LLM is adversarial today, but because (a) prompt injection through user messages or retrieved corpus entries is a real escalation path, and (b) the script can run for hours, eat memory, write anywhere, or call any API if you let it. The sandbox is what makes the whole agent shippable.

Defense layers (you want all of them)

Layer	What it catches
Process isolation (subprocess per run)	Crashes, hangs, leaked DWSIM state
OS resource limits (rlimits / Job Objects)	CPU loops, memory bombs, fork bombs, file-descriptor exhaustion
Network deny (firewall rule on worker UID, or `--net=none`)	Exfiltration, supply-chain calls
Filesystem confinement (chroot / bind-mount / container)	Reads of `~/.aws/credentials`, writes outside run dir
Restricted importer + stripped builtins	Casual `import os`, `open()`, `exec("...")` chains
Solver timeout (in the kit + signal-based hard ceiling)	Unconverged recycles spinning forever

Python-level restrictions alone have been broken many times — they're belt, not suspenders. The OS-level isolation is what actually protects you.

Host orchestrator

```
# executor.py — runs in the agent host. One subprocess per run, no reuse.

import json, os, resource, subprocess, time, uuid
from dataclasses import dataclass
from pathlib import Path

RUNS_ROOT          = Path("/var/run/dwsim-agent")
WORKER             = Path(__file__).parent / "_worker.py"
WALL_TIMEOUT_S     = 180          # hard kill from the host
SOLVER_TIMEOUT_S   = 120          # passed to the worker; must be < WALL
CPU_TIMEOUT_S      = 150
MEM_LIMIT_MB       = 4096

@dataclass
class ExecutorResult:
    envelope: dict
    stdout: str
    stderr: str
    wall_ms: int

def run_script(script: str, label: str) -> ExecutorResult:
    run_id  = f"r_{uuid.uuid4().hex[:6]}"
    run_dir = RUNS_ROOT / run_id
    run_dir.mkdir(parents=True, mode=0o750)
    (run_dir / "script.py").write_text(script)

    request = {
```

```
        "run_id":           run_id,
        "label":            label,
        "script_path":      str(run_dir / "script.py"),
        "out_path":         str(run_dir / "envelope.json"),
        "solver_timeout_s": SOLVER_TIMEOUT_S,
    }

    started = time.monotonic()
    try:
        proc = subprocess.run(
            ["python", "-I", "-S", str(WORKER)],   # -I isolates, -S no
site
            input=json.dumps(request),
            capture_output=True, text=True,
            timeout=WALL_TIMEOUT_S,
            cwd=run_dir,
            env=_minimal_env(run_dir),
            preexec_fn=_apply_rlimits,             # POSIX
            # On Linux, prefer running this whole thing in a container or
            # under systemd-run --user=sandbox --
property=PrivateNetwork=yes.
        )
    except subprocess.TimeoutExpired:
        return ExecutorResult(
            envelope=_timeout_envelope(run_id),
            stdout="", stderr="wall timeout",
            wall_ms=int((time.monotonic()-started)*1000),
        )

    wall_ms = int((time.monotonic()-started)*1000)

    # Worker may have crashed before writing the envelope (segfault from
    # pythonnet/Mono, OOM kill, etc.). Synthesize one in that case.
    out = run_dir / "envelope.json"
    if proc.returncode != 0 or not out.exists():
        envelope = _crash_envelope(run_id, proc)
    else:
        envelope = json.loads(out.read_text())

    return ExecutorResult(envelope, proc.stdout, proc.stderr, wall_ms)


def _apply_rlimits():
    resource.setrlimit(resource.RLIMIT_CPU,    (CPU_TIMEOUT_S,
CPU_TIMEOUT_S))
    resource.setrlimit(resource.RLIMIT_AS,     (MEM_LIMIT_MB << 20,
MEM_LIMIT_MB << 20))
    resource.setrlimit(resource.RLIMIT_CORE,   (0, 0))           # no core
dumps
    resource.setrlimit(resource.RLIMIT_NPROC,  (32, 32))         # cap
forks
    resource.setrlimit(resource.RLIMIT_NOFILE, (256, 256))
    os.setsid()                                                  # own
process group


def _minimal_env(run_dir):
    # Whitelist, not blacklist. Anything not listed is gone.
    return {
        "PATH":         "/usr/bin:/bin",
        "HOME":         str(run_dir),
```

```
        "TMPDIR":       str(run_dir),
        "DWSIM_PATH":   os.environ["DWSIM_PATH"],
        "LD_LIBRARY_PATH": os.environ.get("MONO_LIB", ""),
        # Never pass: API keys, AWS_*, GITHUB_*, ANTHROPIC_*, etc.
    }


def _crash_envelope(run_id, proc):
    # returncode -11 is SIGSEGV; common when DWSIM/Mono blows up
    kind = ("solver_segfault" if proc.returncode == -11
            else "worker_crash")
    return {
        "status":      "error",
        "run_id":      run_id,
        "error_class": kind,
        "issues":      [f"worker exited rc={proc.returncode}"],
        "stderr_tail": proc.stderr[-2000:],
        "hint":        ("DWSIM crashed mid-solve. Often caused by an "
                        "ill-posed recycle or a property package mismatch.
"
                        "Try simpler stoichiometry or a different
package."),
    }
```

Worker — runs inside the sandbox

```
# _worker.py — assumes the script is hostile. Trusts only the host (via
stdin).

import json, signal, sys, traceback
from pathlib import Path

# Imports the script may use. Nothing else.
ALLOWED_IMPORT_ROOTS = {"dwsim_kit"}


class GuardedImporter:
    """Reject any import outside the allow-list. Inserted AFTER dwsim_kit
       has finished pulling in its own .NET assemblies via pythonnet, since
       those imports are not safe to do under the guard."""
    def find_module(self, name, path=None):
        top = name.split(".")[0]
        if top in ALLOWED_IMPORT_ROOTS:
            return None        # fall through to the normal import
machinery
        raise ImportError(f"sandbox: import of {name!r} is not allowed")


def main():
    req      = json.loads(sys.stdin.read())
    run_id   = req["run_id"]
    script   = Path(req["script_path"]).read_text()
    out_path = Path(req["out_path"])
    solver_t = req["solver_timeout_s"]

    # 1. Warm up the kit (which pulls in pythonnet/CLR/DWSIM assemblies)
    #    BEFORE the import guard goes up — otherwise the guard blocks them.
    import dwsim_kit

    # 2. Lock down further imports.
```

```
    sys.meta_path.insert(0, GuardedImporter())

    # 3. Hard solver-timeout fence on top of the kit's own.
    def _timeout(*_):
        raise TimeoutError(f"solver exceeded {solver_t}s")
    signal.signal(signal.SIGALRM, _timeout)
    signal.alarm(solver_t + 10)

    # 4. Strip builtins. open() is gone — kit owns all legitimate I/O.
    ns = {
        "__name__":     "__main__",
        "__builtins__": _safe_builtins(),
        "dwsim_kit":    dwsim_kit,
    }

    try:
        exec(compile(script, "<llm-script>", "exec"), ns)
        result = ns.get("result")
        if result is None:
            raise RuntimeError(
                "script did not assign `result = sim.solve()` to a top-
level name")
        envelope = _success_envelope(run_id, result)
    except Exception as e:
        envelope = _error_envelope(run_id, e)

    out_path.write_text(json.dumps(envelope))


def _safe_builtins():
    import builtins
    allow = {
        "abs", "all", "any", "bool", "dict", "enumerate", "float",
        "int", "isinstance", "issubclass", "len", "list", "map", "max",
        "min", "print", "range", "repr", "reversed", "round", "set",
        "slice", "sorted", "str", "sum", "tuple", "zip",
        "True", "False", "None",
        "__build_class__", "__import__",   # __import__ flows through
GuardedImporter
    }
    return {k: getattr(builtins, k) for k in allow}


def _success_envelope(run_id, result):
    return {
        "status":         "ok",
        "run_id":         run_id,
        "converged":      result.converged,
        "streams":        result.all_streams(),
        "mass_balance":   result.mass_balance(),
        "energy_balance": result.energy_balance(),
        "dwxmz_path":     result._h.dwxmz_path,
    }


def _error_envelope(run_id, exc):
    from dwsim_kit import KitError
    if isinstance(exc, KitError):       cls = "validation"
    elif isinstance(exc, TimeoutError): cls = "timeout"
    elif isinstance(exc, ImportError):  cls = "sandbox"   # tried to import
forbidden module
```

```
    else:                                cls = "solve"
    return {
        "status":      "error",
        "run_id":      run_id,
        "error_class": cls,
        "issues":      [str(exc)],
        "traceback":   traceback.format_exception_only(type(exc), exc),
        "hint":        _hint_for(cls, exc),
    }


def _hint_for(cls, exc):
    return {
        "validation": "Read the issue text. Fix the SPECIFIC field named; "
                      "do not rewrite the whole script.",
        "timeout":    "Solver did not finish. Suspect an unconverged
recycle "
                      "or an over-specified unit. Add a Recycle unit op or
"
                      "loosen one constraint.",
        "sandbox":    "The script tried to import a forbidden module. Only
"
                      "`dwsim_kit` is available. Remove the offending
import.",
        "solve":      "DWSIM raised during solve. Check that all units have
"
                      "feasible specifications.",
    }[cls]


if __name__ == "__main__":
    main()
```

Platform notes

Linux (the path I'd actually ship on). The above is enough for a prototype. For production, wrap the worker in a container or `bwrap` jail:

```
bwrap \
  --ro-bind / / \
  --bind "$RUN_DIR" "$RUN_DIR" --chdir "$RUN_DIR" \
  --tmpfs /tmp --proc /proc --dev /dev \
  --unshare-net --unshare-ipc --unshare-pid \
  --die-with-parent \
  python -I -S _worker.py
```

That gives you a fresh network namespace (no outbound), private PID namespace, and a read-only root with only the run directory writable.

Windows (where DWSIM is most stable). `resource.setrlimit` is a no-op. Use Job Objects via `pywin32`: `win32job.CreateJobObject` + `SetInformationJobObject` with `JobObjectExtendedLimitInformation` for memory and `JOB_OBJECT_LIMIT_KILL_ON_JOB_CLOSE` so children die with the parent. Run the worker as a separate low-privilege Windows user with no network access via a firewall rule scoped to that SID.

Mono on Linux (the riskiest path). Expect periodic segfaults from pythonnet bridging into Mono. The `_crash_envelope` path is not optional — you will hit it in normal operation. Treat segfault as a structured error class and let the agent retry, capped at maybe one retry per failure to avoid burning budget on a genuinely broken script.

What to nail before shipping

1. **Audit the import guard once.** Write a test suite where you throw 20 known escape attempts at the worker (`__import__("os")`, `().__class__.__base__.__subclasses__()`, etc.) and assert each one comes back as an `error_class: "sandbox"`. Python sandbox bypasses are a folk art — your guard will leak. The OS layer is what makes that survivable.
2. **Decide on per-run vs. pooled workers.** Per-run is simpler and the right default. Only pool if startup cost (DWSIM assembly load is ~2–5s) dominates your latency budget — and if you pool, reset the flowsheet state explicitly between runs and crash-restart any worker that hits an `error_class: "solver_segfault"`.
3. **Cap concurrent runs at the host.** DWSIM Automation isn't a happy multi-tenant citizen. Start with a semaphore at 2–4 concurrent workers per host and measure.
4. **Retain run directories for a bounded window.** Each `run_id` directory contains the script, the envelope, and the `.dwxmz`. Keep them for ~24h so the user can download the DWSIM file from `report_to_user`, then GC. Don't keep them forever — they're cheap forensic gold and expensive storage.
5. **Log what the script tried, separately from what it did.** Especially `error_class: "sandbox"` events — those are your prompt-injection canary. Sudden spike in sandbox-class errors means someone is probing.

Want me to sketch the `find_reference_flowsheet` corpus entry format next (what one canonical process template looks like), or the eval harness (how you measure whether the whole pipeline is getting better)?

Here's the corpus entry format. Two things drive every design choice:

1. The LLM needs to *adapt* the template, not just copy it. "Ammonia plant with green H2" should produce a sensible variant, not a parroted base case. That means the entry has to carry the *reasoning* behind each unit op, not just the parameters.
2. Every entry must be machine-validated. A reference that doesn't actually run is worse than no reference — the agent will faithfully reproduce a broken topology. CI re-runs every template on every DWSIM/kit version bump.

Source format (YAML, human-authored)

```yaml
# corpus/ammonia-synthesis-loop.yaml
id: ammonia-synthesis-loop
name: Ammonia Synthesis Loop (Haber-Bosch, single-pass with recycle)
version: 2026-04-12

keywords:
  - ammonia
  - NH3
  - haber-bosch
  - haber bosch
  - nitrogen fixation
  - synthesis loop
related_templates:
  - smr-syngas-cleanup          # if user wants SMR-fed H2
  - air-separation-unit         # if user wants N2 from air

# ── High-level rationale ──────────────────────────────────────
description: |
  Classic high-pressure ammonia synthesis loop. Per-pass conversion is
  deliberately low (~25%); the recycle loop is what produces industrial
  yield. Iron-based catalyst, 200 bar, ~450°C.

operating_regime:
  pressure_bar:    [150, 300]
  temperature_C:   [350, 500]
  property_package: PengRobinson
  why_property_package: |
    High pressure, mostly non-polar (N2, H2, NH3 in gas phase). PR captures
    vapor-phase non-ideality at synthesis pressures. NRTL/UNIQUAC are for
    liquid-phase polar systems and produce nonsense here. Raoult's law at
    200 bar is wildly wrong — never use it.

compounds:
  - Nitrogen
  - Hydrogen
  - Ammonia

# ── Topology ──────────────────────────────────────────────────
block_diagram: |
  FreshFeed ─┐
             ├─→ M1(Mixer) ─→ C1(Compressor) ─→ R1(Reactor) ─→
  Recycle ───┘

                → E1(Cooler) ─→ V1(Flash) ─┬─→ Ammonia (liquid product)
                                           └─→ Recycle (vapor → M1)

units:
  - tag: M1
    kind: Mixer
```

```yaml
    purpose: Combine fresh feed with unconverted recycle gas.

  - tag: C1
    kind: Compressor
    purpose: Boost loop to synthesis pressure.
    params: { P_out_bar: 200 }
    sensitivity: |
      Higher P → higher equilibrium conversion, but capex grows fast.
      Diminishing returns above ~250 bar.
    typical_range: { P_out_bar: [150, 300] }

  - tag: R1
    kind: ConversionReactor
    purpose: "N2 + 3 H2 → 2 NH3 over Fe catalyst."
    params:
      T_out_C: 450
      conversion: 0.25
      basis: Nitrogen
    reactions:
      - stoich: { Nitrogen: -1, Hydrogen: -3, Ammonia: 2 }
        basis: Nitrogen
    sensitivity: |
      Per-pass conversion is intentionally low. Don't try to push it >30% —
      equilibrium and catalyst deactivation cap it. Overall yield comes
      from the recycle loop, not from squeezing the reactor.
    typical_range:
      T_out_C: [400, 500]
      conversion: [0.15, 0.30]

  - tag: E1
    kind: Cooler
    purpose: Condense ammonia for separation.
    params: { T_out_C: -10 }
    sensitivity: |
      -10°C with industrial refrigeration is standard. Lower T improves
      NH3 recovery but burns refrigeration duty.

  - tag: V1
    kind: Flash
    purpose: Separate liquid ammonia from unreacted gas.
    params: { T_C: -10, P_bar: 200 }
    ports_out: [vapor, liquid]
    notes: "V1.liquid is product. V1.vapor recycles."

connections:
  - [FreshFeed, M1]
  - [M1,        C1]
  - [C1,        R1]
  - [R1,        E1]
  - [E1,        V1]

recycles:
  - { from: V1.vapor, to: M1 }

feeds:
  FreshFeed:
    T_C: 25
    P_bar: 1
    composition: { Nitrogen: 0.25, Hydrogen: 0.75 }
    why_composition: |
      Stoichiometric 1:3 N2:H2 maximises conversion. Industrial feeds run
```

```
      slightly H2-rich (~1:3.05) to compensate for catalyst-bed losses.
    flow_basis: |
      User-specified. If unspecified, assume 1 kg/s mass flow for sizing.

products:
  - { tag: V1.liquid, role: primary, expected_purity_NH3: ">=0.95" }

# ── Verifier hookup ──────────────────────────────────────────────
intent_template:
  feed_streams:    [FreshFeed]
  product_streams: [V1.liquid]
  targets:
    product_purity: [V1.liquid, Ammonia, 0.95]
    unit_setpoint:  [R1, T_out_K, 723.15, 5.0]
    min_yield:      [Ammonia, 0.030]      # mol/s, scales with feed

# ── Failure modes the LLM should be warned about ─────────────────
known_failure_modes:
  - "Omitting the recycle: single-pass yield is ~12%, far below industrial.
     Always include add_recycle(from_='V1.vapor', to='M1')."
  - "Mismatched pressures between Compressor outlet and Flash: causes a
     spurious ΔP the solver fights. Keep V1.P_bar == C1.P_out_bar."
  - "Using Raoult's law or NRTL: composition predictions are garbage at
     200 bar. Stick to PengRobinson or SRK."
  - "Argon build-up in the recycle when feed contains inerts: add a
     Splitter purge between V1.vapor and M1 if user mentions Ar > 0.5%."

# ── Adaptation hints ─────────────────────────────────────────────
variations:
  - name: Green ammonia (electrolytic H2)
    delta: |
      No structural change. FreshFeed is still N2 + H2; the upstream
      H2 source is just declared as electrolysis in the user-facing report.
  - name: SMR-fed loop
    delta: |
      Feed contains CO/CO2/CH4. Add the smr-syngas-cleanup template
upstream
      of M1 (methanator + CO2 absorber + dryer).
  - name: Small-scale / modular
    delta: |
      Lower pressure (~100 bar), accept lower per-pass conversion (~15%),
      use higher recycle ratio. Don't change the topology.

# ── The anchor: a known-good kit script ──────────────────────────
kit_skeleton: |
  from dwsim_kit import Flowsheet

  sim = Flowsheet("ammonia_loop")
  sim.use_compounds(["Nitrogen", "Hydrogen", "Ammonia"])
  sim.use_property_package("PengRobinson")

  sim.add_stream("FreshFeed", T=25, P=1, mass_flow=3600,
                 composition={"Nitrogen": 0.25, "Hydrogen": 0.75})

  sim.add_unit("Mixer",             "M1")
  sim.add_unit("Compressor",        "C1", P_out=200)
  sim.add_unit("ConversionReactor", "R1",
               T_out=450,
               reactions=[{"stoich": {"Nitrogen": -1,
                                      "Hydrogen": -3,
                                      "Ammonia":   2},
```

```
                          "basis": "Nitrogen",
                          "conversion": 0.25}])
  sim.add_unit("Cooler", "E1", T_out=-10)
  sim.add_unit("Flash",  "V1", T=-10, P=200)

  sim.connect("FreshFeed", "M1")
  sim.connect("M1",        "C1")
  sim.connect("C1",        "R1")
  sim.connect("R1",        "E1")
  sim.connect("E1",        "V1")
  sim.add_recycle(from_="V1.vapor", to="M1")

  result = sim.solve()

# ── Validation record (written by CI, not by humans) ──────────────────
validated_on:
  dwsim_version:    "8.7.3"
  kit_version:      "0.4.2"
  last_run_utc:     "2026-04-12T14:22:11Z"
  converged:        true
  mass_closure_pct: 99.7
  product_purity:   { compound: Ammonia, stream: V1.liquid, x: 0.962 }
```

What the LLM actually sees

The tool doesn't dump the YAML. It renders a compact, opinionated view ordered for the LLM's workflow — rationale first, anchor script last. Stripped of fields the LLM doesn't act on (`validated_on`, `version`):

```
# Ammonia Synthesis Loop (Haber-Bosch, single-pass with recycle)
Classic high-pressure ammonia synthesis loop. Per-pass conversion is
deliberately low (~25%); the recycle loop is what produces industrial
yield.
OPERATING REGIME: 150–300 bar, 350–500°C, property package = PengRobinson.
  Why PR: vapor-phase non-ideality at synthesis pressures; NRTL/Raoult give
  nonsense here.
COMPOUNDS: Nitrogen, Hydrogen, Ammonia
TOPOLOGY:
  FreshFeed ─┐
             ├─→ M1(Mixer) ─→ C1(Compressor, 200 bar) ─→ R1(Reactor, 450°C,
25% conv)
  Recycle ───┘
            ─→ E1(Cooler, -10°C) ─→ V1(Flash) ─┬─→ Ammonia liquid
                                               └─→ recycle → M1
RATIONALE PER UNIT:
  • C1: pressure trades off conversion vs. capex; sweet spot 200–250 bar.
  • R1: don't push conversion >30%; equilibrium-capped. Loop yield, not
pass yield.
  • E1: -10°C is the industrial standard; lower T = more recovery, more
refrig duty.
  • V1: keep P matched to C1.P_out, or solver fights a spurious ΔP.
FAILURE MODES TO AVOID:
  • Omitting the recycle (yield collapses to ~12%).
  • Mismatched C1/V1 pressures.
  • Using Raoult or NRTL instead of PR/SRK.
  • Argon build-up if feed has inerts — add a Splitter purge.
VARIATIONS:
  • Green ammonia: no structural change; just upstream H2 source.
  • SMR-fed: prepend smr-syngas-cleanup template.
  • Small-scale: drop to ~100 bar, accept ~15% per-pass, raise recycle
ratio.
INTENT TEMPLATE (call declare_intent with this, adjusted to user's feed
flow):
```

```
  feed_streams:    [FreshFeed]
  product_streams: [V1.liquid]
  targets:
    product_purity: [V1.liquid, Ammonia, 0.95]
    unit_setpoint:  [R1, T_out_K, 723.15, 5.0]
ANCHOR SCRIPT (adapt; do not copy verbatim if the user asks for variants):
  from dwsim_kit import Flowsheet
  sim = Flowsheet("ammonia_loop")
  sim.use_compounds(["Nitrogen", "Hydrogen", "Ammonia"])
  sim.use_property_package("PengRobinson")
  sim.add_stream("FreshFeed", T=25, P=1, mass_flow=3600,
                 composition={"Nitrogen": 0.25, "Hydrogen": 0.75})
  sim.add_unit("Mixer",             "M1")
  sim.add_unit("Compressor",        "C1", P_out=200)
  sim.add_unit("ConversionReactor", "R1", T_out=450,
               reactions=[{"stoich": {"Nitrogen": -1, "Hydrogen": -3,
"Ammonia": 2},
                           "basis": "Nitrogen", "conversion": 0.25}])
  sim.add_unit("Cooler", "E1", T_out=-10)
  sim.add_unit("Flash",  "V1", T=-10, P=200)
  for src, dst in [("FreshFeed","M1"),("M1","C1"),("C1","R1"),
                   ("R1","E1"),("E1","V1")]:
      sim.connect(src, dst)
  sim.add_recycle(from_="V1.vapor", to="M1")
  result = sim.solve()
```

That single payload — under 2k tokens — is enough for the LLM to produce a working ammonia loop on the first run for ~any reasonable user variation.

Retrieval

Start simple:

1. Keyword + alias match over the `keywords` field, with fuzzy fallback. "Haber-Bosch", "ammonia plant", "NH3 production" all resolve to `ammonia-synthesis-loop`. This handles ~80% of cases for a small corpus.
2. Embedding similarity over `name + description + keywords` for the long tail. Only worth adding once the corpus exceeds ~30 entries.
3. Return up to 3 candidates with relevance scores. The LLM picks. Don't auto-pick — when the user says "methanol production", it could plausibly mean methanol synthesis loop, methanol-from-CO2, or methanol distillation; let the LLM disambiguate.

Don't over-engineer. RAG can wait until the corpus is big enough to need it.

Authoring discipline (the part that determines whether this works)

1. One author, multiple reviewers. Templates are gold — the LLM will follow them faithfully, so a wrong template silently breaks every user who asks for that process. Author by a chemical engineer who has actually run the simulation; review by at least one other.
2. No template without a passing CI run. A pre-commit hook runs `kit_skeleton` through the executor and asserts `converged && mass_closure_pct > 99.5 && verifier.passed`. Templates that fail block the merge.
3. `sensitivity` and `why_*` fields are mandatory. They're the difference between a template the LLM can adapt and a template it can only parrot. Reviewers should reject entries with empty rationale fields.

4. `known_failure_modes` is harvested from real eval failures. When your eval set surfaces a failure mode the LLM keeps hitting (e.g. omitting recycle), add it to the template. The template is a living document, not a one-time write.
5. Start with 10 templates, not 100. Ammonia, methanol, ethylene cracking, crude distillation, gas dehydration, urea, MEA absorption (CO2 capture), Claus sulfur, methanol distillation, simple flash separation. Those cover most coursework and most early industrial use cases. Add only when eval traces show demand.

What to nail before scaling the corpus

1. One template end-to-end first. Author ammonia, run it through CI, run it through the full agent loop with three user phrasings ("ammonia plant", "NH3 synthesis", "Haber-Bosch loop"), confirm each produces a valid run. Only then write template #2.
2. A template eval set, separate from the agent eval set. Each template gets 3–5 user prompts (base case + 2 variants + 1 adversarial). Pass rate per template is your quality metric. A template that drops below 80% pass rate goes back to the author with the failing traces.
3. A "no template found" path. When `find_reference_flowsheet` returns nothing, the agent shouldn't bluff — it should either tell the user the corpus doesn't have a template and offer to attempt without one (lower confidence) or ask for a known related process. Silent fall-through to "I'll wing it" is the worst outcome.

Want me to sketch the eval harness next (how you measure whether the whole pipeline is improving), or the agent's planning prompt (the actual system prompt + few-shot examples that drive the loop)?

Here's the eval harness. The frame matters: you're not optimizing a model, you're shipping an agent built from many parts — LLM, prompt, kit, templates, hints, verifier, executor. Any of them can regress, and the regressions are often silent (output still looks plausible, just wrong). The harness is what gives you the signal to attribute regressions and ship with confidence.

What you're actually measuring

Four axes. Don't collapse them into a single score — you need to see them separately to debug.

Axis	What it tells you
Correctness	Does the run converge, pass the Verifier, and match expected stream condition
Efficiency	How many tool-call steps and how many tokens to get there?
Topology fidelity	Did the LLM pick the right unit ops and topology (not just *a* converging one)?
Answer quality	Does `report_to_user` actually address what the user asked, including assum

Data model

```python
# eval/task.py

from dataclasses import dataclass, field
from typing import Literal, Optional

Category = Literal["template_base", "template_variant", "adversarial",
                   "out_of_corpus", "regression"]

@dataclass
class StreamExpectation:
    T_K_range:              Optional[tuple[float, float]] = None
    P_Pa_range:             Optional[tuple[float, float]] = None
    mass_flow_kg_s_range:   Optional[tuple[float, float]] = None
    composition_at_least:   dict[str, float] = field(default_factory=dict)
    composition_at_most:    dict[str, float] = field(default_factory=dict)
    phase:                  Optional[str] = None       # "vapor" |
"liquid"

@dataclass
class Expectations:
    must_converge:        bool = True
    must_verify_pass:     bool = True
    max_steps:            int = 8
    max_cost_usd:         float = 0.50
    streams:              dict[str, StreamExpectation] =
field(default_factory=dict)
    must_include_units:   list[str] = field(default_factory=list)    #
["Compressor","Flash"]
    must_include_recycle: bool = False
    answer_must_mention:  list[str] = field(default_factory=list)    #
["recycle","200 bar"]
    answer_must_not_claim:list[str] = field(default_factory=list)    #
["100% conversion"]

@dataclass
class EvalTask:
    id:                   str            # stable
    prompt:               str            # the user message
    category:             Category
```

```
    expected_template_id: Optional[str]  # which template should fire; None
for OOC
    expectations:         Expectations
    tags:                 list[str] = field(default_factory=list)    #
["recycle","high_P"]


# eval/result.py
@dataclass
class EvalResult:
    task_id:        str
    run_id:         str
    success:        bool                  # overall (every must_* satisfied)
    scores:         dict[str, float]      # per-scorer, 0..1
    steps_used:     int
    cost_usd:       float
    template_used:  Optional[str]
    final_envelope: dict                  # last run_flowsheet envelope
    transcript:     list[dict]            # full message history (for
forensics)
    failures:       list[str]             # which scorers failed and why
```

The eval set itself

Split deliberately. Don't mix categories on one leaderboard — they answer different questions.

```
# eval/tasks/ammonia_base.yaml
id:        ammonia_base
category:  template_base
prompt:    "Build me a flowsheet for an ammonia plant."
expected_template_id: ammonia-synthesis-loop
expectations:
  must_converge:        true
  must_verify_pass:     true
  must_include_recycle: true
  must_include_units:   [Mixer, Compressor, ConversionReactor, Cooler,
Flash]
  streams:
    V1.liquid:
      composition_at_least: { Ammonia: 0.90 }
      phase: liquid
  answer_must_mention:    [recycle, "200 bar"]
  answer_must_not_claim:  ["100% conversion"]
  max_steps: 6
  max_cost_usd: 0.30
tags: [recycle, high_pressure, gas_phase]

# eval/tasks/ammonia_green.yaml — variant
id: ammonia_green
category: template_variant
prompt: "Design an ammonia synthesis loop where the hydrogen comes from
electrolysis."
expected_template_id: ammonia-synthesis-loop
expectations:
  must_converge:        true
  must_include_recycle: true
  answer_must_mention:  [electrolysis, recycle]
  # Same numerical expectations as base — green H2 doesn't change topology
  streams:
    V1.liquid:
      composition_at_least: { Ammonia: 0.90 }

# eval/tasks/ammonia_underspec.yaml — adversarial
```

```yaml
id: ammonia_underspec
category: adversarial
prompt: "I want to make ammonia. Help."
expected_template_id: ammonia-synthesis-loop
expectations:
  must_converge: true
  # Key check: did the agent make explicit assumptions in report_to_user?
  answer_must_mention: [assumed, "feed flow", composition]
  max_steps: 8
tags: [missing_specs]
```

Each category serves a different question:

- `template_base` — does the happy path still work? Fast-running, runs on every PR.
- `template_variant` — does the LLM adapt templates correctly?
- `adversarial` — does it handle under-spec, ambiguity, contradiction gracefully?
- `out_of_corpus` — what's the floor when no template fits? (Lower bar — partial credit for picking sensible ops.)
- `regression` — every production failure becomes a permanent task here. The set only grows. Never delete.

The runner

```python
# eval/runner.py — parallel, isolated, deterministic

import asyncio, json, time
from pathlib import Path

async def run_task(task: EvalTask, agent_config: AgentConfig) ->
EvalResult:
    started   = time.monotonic()
    transcript = []
    cost = 0.0
    steps = 0

    async with AgentSession(agent_config) as session:
        async for event in session.run(task.prompt,
max_steps=task.expectations.max_steps):
            transcript.append(event.to_dict())
            if event.kind == "llm_call":
                cost += event.usage.cost_usd
                steps += 1
            if event.kind == "report_to_user":
                final_report = event.payload
                break
        else:
            final_report = {"summary": "(agent did not call
report_to_user)"}

    envelope = session.last_envelope or {}
    scores, failures = score_all(task, envelope, final_report, transcript)

    return EvalResult(
        task_id   = task.id,
        run_id    = envelope.get("run_id", "none"),
        success   = not failures,
        scores    = scores,
        steps_used= steps,
        cost_usd  = cost,
        template_used  = _detect_template_used(transcript),
```

```
        final_envelope = envelope,
        transcript     = transcript,
        failures       = failures,
    )

async def run_suite(tasks: list[EvalTask], cfg: AgentConfig,
                    concurrency: int = 4) -> list[EvalResult]:
    sem = asyncio.Semaphore(concurrency)
    async def _bound(t):
        async with sem:
            return await run_task(t, cfg)
    return await asyncio.gather(*[_bound(t) for t in tasks])
```

Three runner properties that aren't optional:

1. Isolated agent sessions. Each task gets a fresh agent. No memory or cache bleed between tasks — otherwise success on task N leaks into task N+1 and confounds debugging.
2. Full transcript captured. Every LLM call, every tool call, every envelope. This is your forensic record. Cheap to store, irreplaceable when debugging a regression.
3. Deterministic re-runs. Pin model, temperature 0, fixed seeds where available, snapshot the kit + corpus versions. When eval scores move, you need to know whether it's a code change or LLM variance. (You can't eliminate LLM variance entirely; run the eval 3× and report mean + spread.)

The scorers

Each scorer is a small pure function. Keep them composable, never combine them into a megascorer.

```
# eval/scorers.py

def score_convergence(task, env, report, trans) -> tuple[float, list[str]]:
    if task.expectations.must_converge and not env.get("converged"):
        return 0.0, ["did not converge"]
    return 1.0, []

def score_verifier(task, env, report, trans) -> tuple[float, list[str]]:
    if not task.expectations.must_verify_pass:
        return 1.0, []
    v = env.get("verification", {})
    if not v.get("passed", False):
        errs = [f["check"] for f in v.get("findings", []) if f["severity"]
== "error"]
        return 0.0, [f"verifier errors: {errs}"]
    return 1.0, []

def score_streams(task, env, report, trans) -> tuple[float, list[str]]:
    fails = []
    for tag, exp in task.expectations.streams.items():
        s = env.get("streams", {}).get(tag)
        if s is None:
            fails.append(f"stream {tag} missing"); continue
        if exp.T_K_range:
            lo, hi = exp.T_K_range
            if not (lo <= s["T_K"] <= hi):
                fails.append(f"{tag}.T_K={s['T_K']:.1f} not in
{exp.T_K_range}")
        for c, x_min in exp.composition_at_least.items():
            x = s.get("composition", {}).get(c, 0)
            if x < x_min:
```

```
                fails.append(f"{tag}.{c}={x:.3f} < {x_min}")
        if exp.phase and s.get("phase") != exp.phase:
            fails.append(f"{tag}.phase={s.get('phase')} != {exp.phase}")
    return (1.0 - len(fails)/max(len(task.expectations.streams),1)), fails

def score_topology(task, env, report, trans) -> tuple[float, list[str]]:
    units_used = {u["kind"] for u in env.get("units", {}).values()}
    missing = [u for u in task.expectations.must_include_units
               if u not in units_used]
    fails = [f"missing required unit: {u}" for u in missing]
    if task.expectations.must_include_recycle and not env.get("recycles"):
        fails.append("required recycle not present")
    n_req = len(task.expectations.must_include_units) + \
            (1 if task.expectations.must_include_recycle else 0)
    return (1.0 - len(fails)/max(n_req,1)), fails

def score_answer(task, env, report, trans) -> tuple[float, list[str]]:
    """LLM-judge with a tight rubric — keep the judge prompt
deterministic."""
    summary = (report.get("summary") or "").lower()
    fails = []
    for term in task.expectations.answer_must_mention:
        if term.lower() not in summary:
            fails.append(f"answer missing required mention: {term!r}")
    for term in task.expectations.answer_must_not_claim:
        if term.lower() in summary:
            fails.append(f"answer made forbidden claim: {term!r}")
    return 1.0 - len(fails)*0.25, fails

def score_efficiency(task, env, report, trans) -> tuple[float, list[str]]:
    fails = []
    # Hard caps (failures); soft ones go on dashboards, not into pass/fail
    cost = sum(e.get("cost_usd", 0) for e in trans if e.get("kind") ==
"llm_call")
    steps = sum(1 for e in trans if e.get("kind") == "llm_call")
    if cost > task.expectations.max_cost_usd * 2:
        fails.append(f"cost ${cost:.2f} > 2× budget
${task.expectations.max_cost_usd}")
    if steps > task.expectations.max_steps * 2:
        fails.append(f"steps {steps} > 2× budget
{task.expectations.max_steps}")
    return (1.0 if not fails else 0.0), fails

SCORERS = [score_convergence, score_verifier, score_streams,
           score_topology, score_answer, score_efficiency]

def score_all(task, env, report, trans):
    scores, failures = {}, []
    for fn in SCORERS:
        s, f = fn(task, env, report, trans)
        scores[fn.__name__] = s
        failures.extend(f)
    return scores, failures
```

A note on the answer scorer: keyword-matching is a cheap rubric but a real one. Don't reach for an LLM-judge until simple checks demonstrably miss things — LLM-judges introduce their own variance and you spend the rest of your life debating their verdicts. When you do add one, give it a strict rubric ("answer must explicitly state any assumed feed conditions"), few-shot it with 5 labeled examples, and run it at temperature 0.

The report — what makes the harness actually useful

Raw numbers don't drive decisions. Diffs do. Always compare against a frozen baseline.

```
$ eval run --baseline 2026-05-08
EVAL DIFF vs 2026-05-08 (n=47 tasks, 3 reps each)
OVERALL
  Pass rate:         38/47  (81%)    ↓ from 41/47 (87%)
  Mean cost/task:    $0.18            ↑ from $0.14
  Mean steps/task:   4.2              ↑ from 3.8
BY CATEGORY
  template_base        12/12 (100%)   =
  template_variant     11/15  (73%)   ↓ from 14/15 (93%)    ⚠ REGRESSION
  adversarial           8/10  (80%)   =
  out_of_corpus         3/5   (60%)   ↑ from 2/5  (40%)
  regression            4/5   (80%)   ↓ from 5/5 (100%)     ⚠ REGRESSION
REGRESSIONS (was passing → now failing)
  • ammonia_green         — score_topology: missing required unit:
Compressor
  • methanol_recycle      — score_streams:  M2.liquid.Methanol=0.82 < 0.95
  • urea_under_spec       — score_answer:   missing required mention:
'assumed'
  • smr_ng_feed [regr.]   — did not converge
NEW PASSES
  • ammonia_small_scale (was failing on score_topology)
TOP FAILURE MODES (by frequency in transcript hints)
  1. validation: undeclared compound (3 tasks)
  2. solve: unconverged recycle (2 tasks)
  3. timeout (2 tasks)
Forensic transcripts: ./eval/runs/2026-05-12T14-22/
```

Three things this report must do:

1. Surface regressions explicitly. "Was passing, now failing" is the bug list. Sort by it; nothing else.
2. Attribute by category. A drop in `template_variant` while `template_base` holds suggests the LLM is over-fitting to the anchor script (not adapting). A drop in `adversarial` while everything else holds suggests the system prompt got more rigid. The category breakdown points at the layer to fix.
3. Link to transcripts. Every failed task has a one-click path to its full message history. Without that, debugging a 47-task regression is a day; with it, it's an hour.

Where the harness slots into your workflow

```
local dev      → run 5–10 relevant tasks   (~1 min)
PR (CI)        → run template_base + regression   (~5 min)
nightly        → run full suite, post diff to channel   (~30 min)
release gate   → full suite must show no regressions vs. last release
```

Don't run the full suite on every PR. It's slow and noisy enough that engineers will start ignoring failures. Subset on PR, full nightly, gate releases.

Authoring + curation discipline

1. Every reference template gets 3 base + 2 variant + 1 adversarial tasks. Minimum. Templates without eval coverage are reviewed only by humans, and humans miss things.
2. Every production failure becomes a regression task. When a user reports a bad output, the very first action is to file the trace as a frozen task in the `regression` category. The fix is verified by the new task going green. The task never gets removed — that's how you ratchet quality.

3. Hand-author the first ~30 tasks. Don't generate them. Synthetic prompts cluster around how-an-LLM-thinks-a-user-would-ask, not how users actually ask. Use your own user logs once you have them.
4. Lock the baseline explicitly. Don't auto-promote yesterday's run to the baseline. A human looks at the diff and decides "this is the new floor." Otherwise you ratchet downward one bad day at a time.
5. Budget for eval cost. A 47-task suite at $0.18/task × 3 reps × nightly = ~$25/day. That's fine. A 500-task suite at $0.50/task × 3 reps × every PR = something you'll stop running. Keep the on-every-PR subset cheap and the full suite tractable.

What to nail before scaling

1. Get one task end-to-end first. Author `ammonia_base`, write all six scorers, run it, hand-verify each scorer's verdict matches your intuition. Only then write task #2. The scorer logic has subtle bugs that only show up on real outputs.
2. Decide what "success" means at the task level. All-scorers-pass is unforgiving; some-scorers-pass is permissive. I'd start with: `must_converge + must_verify_pass + must_include_recycle` are hard gates; everything else contributes to a score the report shows but doesn't fail on. Tighten over time.
3. Pin the model. When you upgrade Claude Sonnet 4.6 → 4.7, re-baseline explicitly with a labeled run. Otherwise you'll mistake a model improvement (or regression) for a code change.
4. Run-to-run variance is real. 3 reps and report mean is the minimum honest floor. If a task swings between pass and fail across reps with the same code, that's a *signal*, not noise — the agent is on the edge for that task. Either tighten the prompt, the template, or the task's tolerances.
5. The forensic path is the product. Most of the value of an eval harness is not the score — it's the speed with which you can answer "why did this task fail?". Invest in transcript readability (rendered LLM messages, color-coded tool calls, envelope diffs) before you invest in fancier scoring.

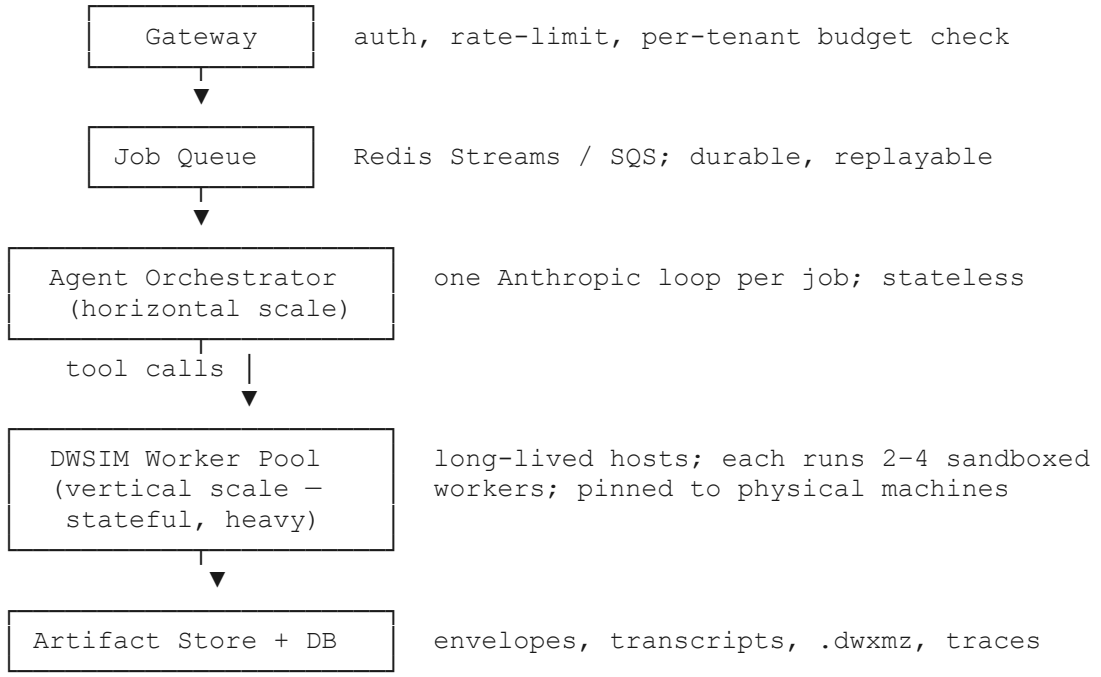
That closes the loop. Phase 1 (reference script) → Phase 2 (kit) → Phase 3 (tools + verifier + executor + corpus) → Phase 4 (eval harness). If you ship those six pieces in that order, each one validating the last, you'll have a working agent before the corpus has more than 5 templates — and you'll know it's working because the harness will show it.

Here's Phase 5. The shift in mindset: until now you've optimized for *does it work*. From here on, you optimize for *does it stay working under load, abuse, drift, and bad days*. Different muscle group. Less code, more operational architecture and trade-offs.

What changes when you go to production

Phase 1–4 concern	Phase 5 equivalent
One user at a time	N concurrent users, isolated
Agent runs locally	Worker pool with queue and backpressure
Cost is your wallet	Cost is per-tenant, budgeted, alerted
Errors print to terminal	Errors are SLOs, dashboards, pages
You rerun manually	Canary, feature flags, instant rollback
Eval set is a folder	Eval set is part of the release gate
Failure mode = annoyance	Failure mode = incident with blast radius

Operational architecture



Two separate scaling stories. The orchestrator is stateless Python — autoscale on CPU/queue depth. The DWSIM workers are stateful, slow to warm, prone to native crashes — treat them like database replicas, not lambdas.

```python
# worker_pool.py — what manages DWSIM hosts

class WorkerHost:
    """One physical/VM host. Runs N concurrent sandboxed workers
    (typically 2-4; DWSIM Automation is not happily multi-tenant)."""
    host_id:        str
    capacity:       int                # max concurrent sandboxes
    in_flight:      int
    consecutive_segfaults: int         # circuit-breaker counter
    last_warm:      float              # assemblies loaded at...
```

```
class WorkerPool:
    def lease(self, run_id: str) -> WorkerHost:
        # 1. Skip hosts with >=3 consecutive segfaults (cooling off)
        # 2. Prefer warm hosts (assemblies loaded) by last_warm desc
        # 3. Fall back to capacity headroom
        # 4. If no host available, push back onto queue with delay
        ...

    def release(self, host: WorkerHost, outcome: Outcome):
        host.in_flight -= 1
        if outcome.was_segfault:
            host.consecutive_segfaults += 1
            if host.consecutive_segfaults >= 3:
                self._quarantine(host)          # drain + restart process
        else:
            host.consecutive_segfaults = 0
```

The segfault circuit breaker is not optional. DWSIM/Mono will randomly nuke a worker host; without quarantine, the orchestrator keeps routing to a host that's about to crash again and you get a cascade.

Observability — the three signals you actually need

Don't instrument everything. Pick the views that drive decisions.

```
# observability/trace.py

# Span hierarchy that's worth its weight:
#   request                          (per user request)
#     ├─ planning                    (LLM calls before first run_flowsheet)
#     ├─ run_flowsheet [N]           (each attempt)
#     │    ├─ sandbox_exec           (subprocess time)
#     │    └─ verify                 (deterministic; should be ~10-50ms)
#     └─ report_to_user

@contextmanager
def span(name: str, **attrs):
    span_id = otel.start_span(name, attrs)
    try:
        yield span_id
    except Exception as e:
        otel.set_status(span_id, "error", str(e))
        raise
    finally:
        otel.end_span(span_id)
```

The dashboards that pay for themselves:

View	Decision it drives
Per-task success rate (7-day, by category)	When to pull a canary, when to ship
Steps-to-success distribution	When hint quality is degrading (right tail growing = LLM looping)
Cost-per-successful-task	When the budget is broken
Verifier-pass-rate among "converged" runs	The single best leading indicator of silent quality loss
DWSIM segfault rate per host	Capacity planning, host quarantine

View	Decision it drives
Sandbox-class errors over time	Prompt-injection canary
p50/p95/p99 latency end-to-end	What you promise users

Pages on:

- Pass rate drop >5 pts over 1h (likely model or prompt regression)
- Verifier-pass among converged drops >10 pts (silent quality loss)
- Sandbox-class error rate >5/min (likely active injection probing)
- Worker-pool capacity headroom <20% sustained (queue is about to back up)

Don't page on individual run failures. Pages on aggregates, not events.

Reliability — failure modes and their responses

```
# reliability/policies.py

class FailureClass(Enum):
    LLM_RATE_LIMIT      = auto()    # retry with backoff; don't burn budget
    LLM_5XX             = auto()    # retry once; fall back to Sonnet if Opus
fails
    SANDBOX_SEGFAULT    = auto()    # retry once on different host
    SOLVER_TIMEOUT      = auto()    # do NOT retry — same script will hang
again
    VERIFIER_REJECTED   = auto()    # the agent loop handles via hints
    SANDBOX_VIOLATION   = auto()    # never retry; alert
    BUDGET_EXCEEDED     = auto()    # never retry; surface to user

RETRY_POLICY = {
    FailureClass.LLM_RATE_LIMIT:    Retry(max=5, backoff="exp",
jitter=True),
    FailureClass.LLM_5XX:           Retry(max=2, backoff="exp",
                                          fallback_model="claude-sonnet-4-
6"),
    FailureClass.SANDBOX_SEGFAULT: Retry(max=1, route="different_host"),
    FailureClass.SOLVER_TIMEOUT:   Retry(max=0),
    FailureClass.SANDBOX_VIOLATION:Retry(max=0, alert="security"),
    FailureClass.BUDGET_EXCEEDED:  Retry(max=0, user_message=True),
}
```

Two principles:

1. Retry only what's retryable. Solver timeouts on the same script will timeout again — retrying burns a worker slot for nothing. Distinguish "transient infrastructure" from "deterministic agent failure."
2. Brownout, don't blackout. When the worker pool is saturated, don't error out — degrade. New requests get a "queued, ~2 min wait" response with a job ID. Premium tenants jump the queue. The system stays *useful* under load, not just up.

Cost & abuse controls

```
# limits/tenant.py

@dataclass
class TenantBudget:
    daily_usd:        float
    daily_runs:       int
```

```
    concurrent_runs:  int
    max_steps_per_run:int = 10          # absolute ceiling, agent_config
caps lower

class GatewayCheck:
    def admit(self, tenant: Tenant, request: Request) -> AdmitResult:
        if self.daily_spend(tenant) >= tenant.budget.daily_usd:
            return Deny("daily budget exhausted",
retry_after=midnight_utc())
        if self.in_flight(tenant) >= tenant.budget.concurrent_runs:
            return Queue("at concurrency limit")
        if self.recent_sandbox_violations(tenant, window="1h") > 3:
            return Deny("flagged for review", alert="abuse")
        return Admit()
```

Per-tenant budget is the single most important production control after the sandbox. Without it, one bad prompt costs $200 and you find out on the bill.

Versioning & reproducibility

Every artifact in a run is the product of several versioned things. Pin them all, log them all, every time.

```
@dataclass
class RunManifest:
    run_id:           str
    started_at:       str
    tenant_id:        str
    # Pinned versions — recorded with the run, never inferred
    model:            str          # "claude-opus-4-7-20260301"
    system_prompt:    str          # SHA-256 of the prompt
    tools_schema:     str          # SHA-256
    kit_version:      str          # "dwsim_kit==2.4.1"
    corpus_version:   str          # git SHA of reference flowsheets
    dwsim_version:    str          # "8.7.6"
    sandbox_version:  str
    eval_baseline:    str          # which baseline this run is gated
against
```

Reproducibility means: any run from the last 90 days can be re-executed and produce a functionally equivalent output. When a user says "the run from last Tuesday was good, the one today is wrong," you can diff manifests and find the changed input.

Data lifecycle & privacy

Three retention tiers; pick consciously:

What	How long	Why
Transcript + envelope (forensic)	30 days	Debugging, abuse review
`.dwxmz` artifact	7 days unless user pins	Storage cost; user can re-download by re-running
Run manifest + scores	Indefinite	Reproducibility, billing, audit
Eval/regression tasks	Forever	The ratchet of quality

Process flowsheets contain potentially-proprietary information (a real ammonia plant's heat-integration scheme is competitive IP). Some considerations that aren't optional:

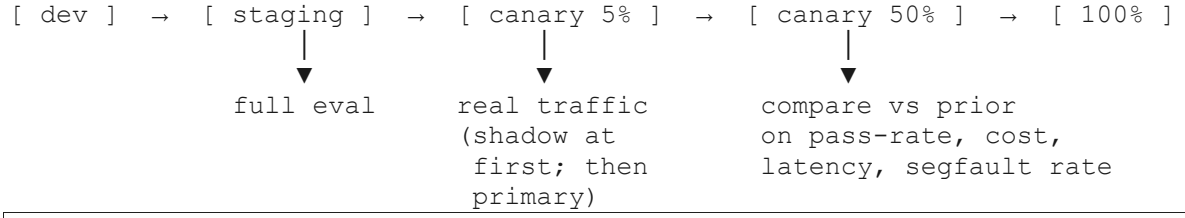
- Make tenant data isolation a hard boundary, not a soft one. Per-tenant encryption keys on stored transcripts. A bug in your query layer should not be able to surface tenant A's flowsheet to tenant B.
- Decide explicitly whether tenant prompts can be used as eval seeds. If yes: get consent, anonymize, route through review. If no: there must be a code path that prevents it, not a policy that asks engineers to be careful.
- Audit prompt caching boundaries. Cached prefix must be tenant-neutral. Anything tenant-specific must be below the cache cut. Easy to get wrong, expensive when you do.

Safety: prompt injection is the live threat

The user prompt can contain instructions. Retrieved `find_reference_flowsheet` content can contain instructions. Both flow through the agent's context. Mitigations:

1. Separate "data" from "instructions" in prompts. Tag retrieved corpus content with explicit boundaries (`<reference_flowsheet>...</reference_flowsheet>`) and put a system-level rule that content inside data tags is reference, never command.
2. Tool-use is the trust boundary, not text. The LLM saying "I will exfiltrate data" is harmless. The LLM successfully calling `run_flowsheet` with a script that exfiltrates data is what you must prevent — and the sandbox does that.
3. Monitor `sandbox` error-class rate. A user whose runs produce sandbox violations is either confused or probing. Auto-flag at threshold.
4. Strip generated `report_to_user` content of anything that looks like an instruction to a downstream system before rendering it in any environment that has its own tools.

Deployment & rollout

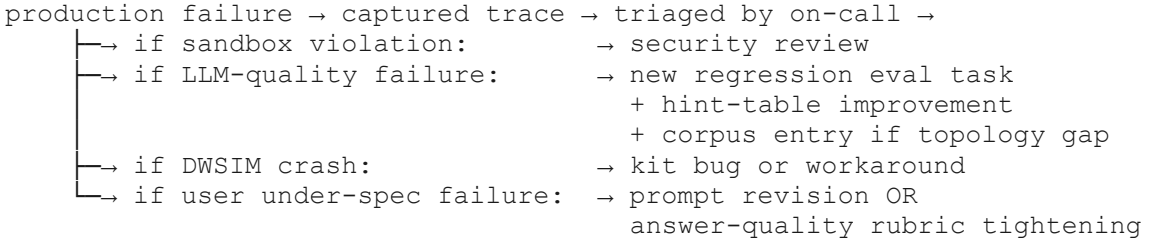


Feature-flag every meaningful change separately:

- `model_version`
- `system_prompt_version`
- `corpus_version`
- `tool_schema_version`
- `verifier_rules_version`

When the canary regresses, you need to flip *one* flag back, not redeploy the world. Don't bundle changes — bundling guarantees that when something breaks, you don't know what.

The feedback loop closes here



Every incident has to end with either (a) a new eval task that would have caught it, or (b) an explicit decision that it's not worth catching. No third option. Otherwise you'll regress in the same way twice.

What to nail before launch — the actual checklist

1. One incident drill, done for real. Kill a DWSIM worker mid-run during staging. Confirm the user sees a useful error, the queue keeps moving, and the segfault counter increments. If any of those fail, you're not ready.
2. Budget enforcement is end-to-end testable. Set a tenant's daily limit to $0.05, run two prompts, second one must deny cleanly. If you can't write that test, you don't have budget enforcement — you have budget hope.
3. Rollback is one command. Practice it. Time it. If rollback takes longer than 5 minutes you will not use it during an incident and you'll choose to "ride it out" instead. That's how incidents become outages.
4. Eval as release gate is wired in CI. No human-in-the-loop decision to "promote anyway because we're behind." That decision exists; just make it expensive enough that it isn't the default.
5. You can answer "what did the agent do for run X?" in under 60 seconds. Open trace, see plan, see each `run_flowsheet` attempt, see Verifier findings, see final report. If retrieving that takes a SQL query, you'll skip it under load and debug blind.
6. Per-tenant data isolation has been adversarially tested. Not "we wrote the code to filter by tenant_id" — but "someone tried to break out of their tenant and failed." This is the litmus test that separates products from prototypes.
7. An obvious "this is wrong" button in the UI. Users will spot bad flowsheets you didn't catch. Capture their feedback into the regression queue automatically. This single feature does more for long-term quality than any model upgrade.
8. A decision, in writing, about what you don't catch. Not every failure is worth a regression task. Write down which failure classes are explicitly out-of-scope (e.g., "we don't support fluidized bed reactors in v1; we route to a graceful refusal"). Make refusal as much of a first-class outcome as success.

That's the production hardening sketch. The path through all five phases — script → kit → agent → verifier → executor → corpus → eval → harness → production controls — is roughly 6–9 months of focused work for a small team. The thing to internalize is that Phases 1–4 build the product; Phase 5 keeps it. Skipping Phase 5 isn't a shortcut, it's a deferred bill that compounds.